

CS336 作业 1（基础）：构建 Transformer LM

版本 1.0.6

CS336 教职员工

2025年春季

1. 作业概述

在本作业中，您将从头开始构建训练标准 Transformer 语言模型 (LM) 所需的所有组件，并训练一些模型。

您将实施什么

1. 字节对编码 (BPE) 分词器 (§ 2)
2. Transformer 语言模型 (LM) (§ 3)
3. 交叉熵损失函数和 AdamW 优化器 (§ 4)
4. 训练循环，支持序列化和加载模型及优化器状态 (§ 5)

你将运行什么

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 在数据集上运行训练好的分词器，将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer 语言模型。
4. 使用训练好的 Transformer LM 生成样本并评估困惑度。
5. 在 OpenWebText 上训练模型，并将您获得的困惑度提交到排行榜。

你可以使用我们希望您从零开始构建这些组件。特别是，您可能需要……不是使用以下任何定义 `torch.nn`，`torch.nn.functional`，或者 `torch.optim` 以下情况除外：

- `torch.nn` 参数
- 容器类 `torch.nn`（例如，模块、模块列表、顺序、ETC。）¹
- 这 `torch.optim` 优化器基类

您可以使用任何其他 PyTorch 定义。如果您想使用某个函数或类，但不确定是否允许，请随时在 Slack 上提问。如有疑问，请考虑使用该函数或类是否会违背本次作业“从零开始”的理念。

¹有关完整列表，请参阅 [PyTorch.org/docs/stable/nn.html#containers](https://pytorch.org/docs/stable/nn.html#containers)。

关于人工智能工具的声明允许使用 ChatGPT 等语言模型提示低级编程问题或高级语言模型概念问题，但禁止直接使用语言模型来解决问题。

我们强烈建议您在完成作业时禁用 IDE 中的 AI 自动补全功能（例如 Cursor Tab、GitHub CoPilot）（但非 AI 自动补全功能，例如自动补全函数名称，完全没问题）。我们发现 AI 自动补全功能会大大降低您深入理解学习内容的能力。

代码看起来是这样的所有作业代码以及本文均可在 GitHub 上找到：

github.com/stanford-cs336/assignment1-basics

请git 克隆该存储库。如有任何更新，我们将通知您，以便您可以……git pull获取最新消息。

- 1.cs336_basics/*:这里是编写代码的地方。请注意，这里没有任何代码——您可以从头开始编写任何您想要的东西！
- 2.adapters.py: 你的代码必须具备一系列功能。对于每个功能（例如，缩放点积注意力），请填写其实现（例如，运行缩放点积注意力）只需调用您的代码即可。注意：您对以下内容的更改 adapters.py不应包含任何实质性逻辑；这只是粘合代码。
- 3.test_*.py: 这包含了你必须通过的所有测试（例如，test_scaled_dot_product_attention），这将调用在以下位置定义的钩子：adapters.py。请勿编辑测试文件。

如何提交您需要向 Gradescope 提交以下文件：

- writeup.pdf: 请回答所有书面问题。请用电脑打字作答。
- code.zip: 包含你编写的所有代码。

要将作品提交到排行榜，请提交 PR 至：

github.com/stanford-cs336/assignment1-basics-leaderboard

参见README.md请在排行榜库中查看详细的提交说明。

哪里可以获取数据集本次作业将使用两个预处理数据集：TinyStories [Eldan and Li, 2023] 和 OpenWebText [Gokaslan et al., 2019]。这两个数据集都是单个的大型纯文本文件。如果您是和同学一起完成这项作业，可以在 / 找到这些文件。数据任何非头节点机器。

如果您在家跟着教程操作，可以使用命令提示符下载这些文件。README.md。

低资源/缩减规模提示：初始化

在本课程的作业讲义中，我们会提供一些建议，帮助学员在GPU资源较少或没有GPU资源的情况下完成部分作业。例如，我们有时会建议……**缩小**您的数据集或模型大小，或者如何解释如何在 macOS 集成 GPU 或 CPU 上运行训练代码。您会在蓝色方框（例如这个）中找到这些“低资源使用技巧”。即使您是斯坦福大学的注册学生，可以访问课程专用机器，这些技巧也能帮助您更快地迭代并节省时间，因此我们建议您阅读它们！

低资源/缩减技巧：作业 1 在 Apple Silicon 或 CPU 上

利用我们提供的解决方案代码，我们可以在配备 36GB 内存的 Apple M3 Max 芯片上训练一个语言模型，使其生成相当流畅的文本；使用 Metal GPU (MPS) 时，只需不到 5 分钟；而使用 CPU 时，则大约需要 30 分钟。如果您对这些术语不太熟悉，也无需担心！您只需知道，如果您拥有一台配置较高的笔记本电脑，并且您的实现正确高效，那么您就能训练出一个小型语言模型，生成流畅度尚可的简单儿童故事。

稍后在作业中，我们将解释如果您使用的是 CPU 或 MPS，需要进行哪些更改。

2 字节对编码 (BPE) 分词器

在本作业的第一部分，我们将训练并实现一个字节级字节对编码（BPE）分词器[Sennrich et al., 2016, Wang et al., 2019]。具体来说，我们将任意（Unicode）字符串表示为一个字节序列，并使用该字节序列训练我们的BPE分词器。之后，我们将使用该分词器将文本（字符串）编码为词元（整数序列），用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它将字符映射到整数。代码点截至 Unicode 16.0（2024 年 9 月发布），该标准定义了 168 种文字的 154,998 个字符。例如，字符“s”的代码点为 115（通常表示为 115）。U+0073，在哪里 U+ 是一个约定俗成的前缀，0073（十六进制为 115），以及字符“牛”代码点为 29275。在 Python 中，您可以使用 `ord()` 将单个 Unicode 字符转换为其整数表示形式的函数。`chr()` 该函数将整数 Unicode 代码点转换为包含相应字符的字符串。

```
>>> 秩序 ('牛')
29275
>>> 克 (29275)
'牛'
```

问题 (unicode1):了解Unicode (1分)

(a) 哪个Unicode字符表示`chr(0)`返回？ **交付成果**

一句话回复。

(b) 这个字符的字符串表示形式是怎样的（`__repr__()`）与印刷版本有何不同？

交付成果一句话回复。

(c) 当这个字符出现在文本中时会发生什么？在 Python 解释器中尝试以下代码，看看是否符合你的预期，这可能会有所帮助：

```
>>> 克 (0)
>>> 打印 (克 (0) )
>>> “这是一次测试。+克 (0) + “细绳”
>>> 打印 (这是一次测试。+克 (0) + “细绳” )
```

交付成果一句话回复。

2.2 Unicode 编码

虽然 Unicode 标准定义了字符到代码点（整数）的映射，但直接基于 Unicode 代码点训练分词器并不实际，因为词汇量会非常庞大（约 15 万个条目）且稀疏（因为许多字符非常罕见）。因此，我们将使用 Unicode 编码，它将 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32，其中 UTF-8 是互联网上最主要的编码（超过 98% 的网页都使用 UTF-8）。

要将 Unicode 字符串编码为 UTF-8，我们可以使用以下方法：`encode()` Python 函数。用于访问 Python 的底层字节值。字节对于该对象，我们可以对其进行迭代（例如，调用）。列表（`list()`）。最后，我们可以使用解码（`decode()`）将 UTF-8 字节字符串解码为 Unicode 字符串的函数。

```

>>> 测试字符串= “你好! こんにちは!”
>>> utf8_encoded=测试字符串。编码("utf-8")
>>> 打印(utf8编码)
嗨! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf! '
>>> 打印(类型(utf8_encoded))
<class 'bytes'>
>>> # 获取编码字符串的字节值 (0 到 255 之间的整数)。
>>> 列表(utf8编码)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129, 161, 227, 129, 175, 33]

>>> # 一个字节并不一定对应一个Unicode字符!
>>> 打印(镜头(测试字符串))
13
>>> 打印(镜头(utf8_encoded)) 23

>>> 打印(utf8_encoded。解码("utf-8")) 你
好! こんにちは!

```

通过将 Unicode 码位转换为字节序列（例如，通过 UTF-8 编码），我们实际上是将码位序列（0 到 154,997 范围内的整数）转换为字节值序列（0 到 255 范围内的整数）。长度为 256 的字节词汇表是*很多*更易于处理。使用字节级分词时，我们无需担心词汇表外的词元，因为我们知道……*任何*输入文本可以表示为 0 到 255 之间的整数序列。

问题 (unicode2):Unicode编码 (3分)

- (a) 为什么我们更倾向于使用 UTF-8 编码的字节来训练分词器，而不是使用 UTF-16 或 UTF-32 编码的字节？比较这些编码对不同输入字符串的输出结果可能会有所帮助。

交付成果用一到两句话回答。

- (b) 考虑以下（错误的）函数，该函数旨在将 UTF-8 字节字符串解码为 Unicode 字符串。为什么这个函数是错误的？请提供一个会产生错误结果的输入字节字符串示例。

定义`decode_utf8_bytes_to_str_wrong`（字节串：字节）：
返回""。加入（[字节（[b]）。解码（"utf-8"）为了b在字节串]）

```

>>> decode_utf8_bytes_to_str_wrong( “你好” 。编码("utf-8") ) '你
好'

```

交付成果：一个示例输入字节字符串，其中`decode_utf8_bytes_to_str_wrong`产生错误输出，并用一句话解释函数错误的原因。

- (c) 给出一个不解码成任何 Unicode 字符的双字节序列。**交付成果**举个例子，并用一句话解释。

2.3 子词分词

虽然字节级分词可以缓解词级分词器面临的词汇表外问题，但将文本分词成字节会导致输入序列过长。这会减慢模型训练速度，因为……

在词级语言模型中，一个包含 10 个单词的句子可能只有 10 个标记，但在字符级模型中，它可能有 50 个或更多标记（取决于单词的长度）。处理这些更长的序列需要在模型的每个步骤中消耗更多的计算资源。此外，基于字节序列的语言建模也十分困难，因为更长的输入序列会在数据中产生长期依赖关系。

子词分词介于词级分词器和字节级分词器之间。请注意，字节级分词器的词汇表包含 256 个条目（字节值为 0 到 255）。子词分词器牺牲了较大的词汇表容量，换取了对输入字节序列更好的压缩。例如，如果字节序列 b'the' 这种情况经常出现在我们的原始文本训练数据中，给它分配一个词汇表条目会将这个 3 个标记的序列简化为一个标记。

我们如何选择要添加到词汇表中的子词单元？Sennrich 等人 [2016] 提出使用字节对编码 (BPE; Gage, 1994)，这是一种压缩算法，它迭代地将出现频率最高的字节对替换（“合并”）为一个新的、未使用的索引。请注意，该算法将子词标记添加到词汇表中，以最大程度地压缩输入序列——如果一个词在输入文本中出现足够多次，它将被表示为一个单独的子词单元。

使用基于字节序列扩展 (BPE) 构建的词汇表的子词分词器通常被称为 BPE 分词器。在本作业中，我们将实现一个字节级 BPE 分词器，其中词汇表项是字节或合并的字节序列，这在处理词汇表外的词项和控制输入序列长度方面兼具两者的优势。构建 BPE 分词器词汇表的过程称为“训练”BPE 分词器。

2.4 BPE分词器培训

BPE 分词器训练过程包括三个主要步骤。

词汇初始化分词器词汇表是字节串标记到整数 ID 的一一对应关系。由于我们训练的是字节级 BPE 分词器，因此初始词汇表就是所有字节的集合。由于有 256 个可能的字节值，因此初始词汇表的大小为 256。

预标记化一旦有了词汇表，理论上你可以统计文本中相邻字节出现的频率，并从出现频率最高的字节对开始合并它们。然而，这种方法计算量非常大，因为每次合并都需要对整个语料库进行一次完整的遍历。此外，直接在语料库中合并字节可能会导致生成的标记仅在标点符号上有所不同（例如，狗！对比狗。）。即使这些词条可能具有很高的语义相似性（因为它们仅在标点符号上有所不同），它们也会获得完全不同的词条 ID。

为了避免这种情况，我们**预标记化**语料库。您可以将其视为对语料库进行粗粒度分词，以帮助我们统计字符对出现的频率。例如，单词“文本”可能存在出现 10 次的前缀标记。在这种情况下，当我们统计字符“t”和“e”相邻出现的次数时，会发现单词“text”中“t”和“e”相邻出现，我们可以直接将它们的计数加 10，而无需遍历整个语料库。由于我们训练的是字节级 BPE 模型，因此每个前缀标记都表示为一个 UTF-8 字节序列。

Sennrich 等人 [2016] 的原始 BPE 实现通过简单地按空格分割（即，`s.split(" ")`）。相比之下，我们将使用基于正则表达式的预分词器（GPT-2 使用；Radford 等人，2019）。github.com/openai/tiktoken/pull/234/files:

```
>>> 拍=r'(?:[sdmt]|ll|ve|re)?\p{L}+|\p{N}+|(?^\s\p{L}\p{N}+)|s+(?!S)\s+''''
```

使用此预分词器交互式地拆分一些文本可能有助于更好地了解其运行情况：

```
>>> # 需要`regex`包
>>> 进口正则表达式作为关于
>>> 关于。查找所有(PAT, “一些我需要预先分词的文本”) ['some', 'text',
'that', 'i', 'll', 'pre', '-', 'tokenize']
```

但是，在代码中使用它时，你应该使用关于。查找器为了避免在构建从预标记到其计数的映射时存储预标记的单词。

计算 BPE 合并现在我们已经将输入文本转换为预标记，并将每个预标记表示为 UTF-8 字节序列，接下来就可以计算 BPE 合并（即训练 BPE 分词器）。简而言之，BPE 算法会迭代地统计每一对字节，并找出出现频率最高的字节对（“A”、“B”）。然后，每次出现这最频繁的字节对（“A”、“B”）时，都会被合并。合并即，替换为新的标记“AB”。这个合并后的新标记被添加到我们的词汇表中；因此，BPE 训练后的最终词汇表大小为初始词汇表的大小（在本例中为 256），加上训练期间执行的 BPE 合并操作次数。为了提高 BPE 训练的效率，我们不考虑跨越预标记边界的词对。²计算合并时，确定性地打破配对频率相同的条件 *倾向于选择词典顺序上更靠前的词对*例如，如果（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）这几对出现频率最高，我们就合并（“BA”，“A”）：

```
>>> 最大限度((( "一个", "B"), ("一个", "C"), ("B", "ZZ"), ("BA", "一个")
))) ('BA', 'A')
```

特殊代币通常，某些字符串（例如，<|endoftext|>）用于编码元数据（例如，文档之间的边界）。在对文本进行编码时，通常需要将某些字符串视为“特殊标记”，这些标记永远不应被拆分为多个标记（即，始终保留为单个标记）。例如，序列结束字符串 <|文本结束|>应该始终保留为单个词元（即单个整数 ID），这样我们才能知道何时停止从语言模型生成词元。这些特殊词元必须添加到词汇表中，以便它们具有对应的固定词元 ID。

Sennrich 等人 [2016] 的算法 1 包含一种效率较低的 BPE 分词器训练实现（基本上遵循我们上面概述的步骤）。作为初步练习，实现并测试此函数可能有助于检验您对该算法的理解。

例子（bpe_example）：BPE 培训示例

以下是 Sennrich 等人 [2016] 提供的一个示例。考虑一个包含以下文本的语料库。

低 低 低 低 低
最低 最低 最宽 最宽 最新 最新 最新 最新

词汇表有一个特殊的标记 <|endoftext|>。

词汇我们使用特殊标记 <| 初始化词汇表。文本结束|>以及 256 字节的值。

预标记化为了简化说明并专注于合并过程，本例中我们假设预分词仅以空格为分割依据。预分词和计数后，即可得到频率表。

{最低：5，最低：2，最宽：3，最新：6}

²需要注意的是，原始的 BPE 公式 [Sennrich et al., 2016] 规定包含词尾标记。我们在训练字节级 BPE 模型时没有添加词尾标记，因为模型词汇表中包含了所有字节（包括空格和标点符号）。由于我们显式地表示了空格和标点符号，因此学习到的 BPE 合并结果自然会反映这些词边界。

为了方便起见，我们可以将其表示为词典[元组[字节],整数]例如{(低的) : 5 ...}。请注意，即使是单个字节也是一个字节Python中的对象。没有字节在Python中，要表示单个字节，必须输入`int`或`int`，就像没有`int`或`int`一样。字符在Python中输入表示单个字符的字符串。

合并我们首先查看每一对连续的字节，并计算它们出现的单词的频率 {lo:7、现在: 7、我们: 8、呃: 2, wi: 3, ID: 3, de:3, es: 9、英石: 9、ne: 6、呃: 6}。这对 ('es') 和 ('英石') 这两个词并列，所以我们取字典序较大的那对， ('英石')。然后将合并前缀标记，最终得到{(低的) : 5, (降低) : 2, (w,i,d,e,st):3, (最新) : 6}。

在第二轮中，我们看到 (e, st) 是最常见的组合 (数量为 9)，我们将合并到 {(低的) : 5, (降低) : 2, (w,i,d,est):3, (最新) : 6}。继续这样做，我们最终得到的合并序列将是['s t'、'e st'、'o w'、'l ow'、'w est'、'n e'、'ne west'、'w i'、'w i d'、'w i d est'、'low e'、'lowe r']。

如果我们进行 6 次合并，我们将得到 ['s t'、'e st'、'o w'、'l ow'、'w est'、'n e'] 我们的词汇元素将是 [<|endoftext|>, [...256 字节字符], st, est, ow, low, west, ne]。

凭借这些词汇和合并规则，这个词最新会被标记化为 [东北, 西]。

2.5 尝试使用 BPE 分词器进行训练

让我们在 TinyStories 数据集上训练一个字节级 BPE 分词器。数据集的查找/下载说明请参见第 1 节。在开始之前，我们建议您先浏览一下 TinyStories 数据集，以便了解数据内容。

并行化预标记化你会发现，主要的瓶颈在于预标记化步骤。你可以通过使用内置库并行化代码来加快预标记化速度。多进程。具体来说，我们建议在并行实现预分词时，将语料库分块，并确保分块边界位于特定词元的开头。您可以直接使用以下链接中的初始代码来获取分块边界，然后利用这些边界将工作分配到各个进程中：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块方法始终有效，因为我们永远不希望跨越文档边界进行合并。就本次作业而言，您可以始终按此方式拆分。不必担心收到一个非常大的语料库，其中不包含 <|endoftext|>。

在预标记化之前移除特殊标记在运行预标记化之前（使用正则表达式模式）关于。查找器），你应该从语料库（或数据块，如果使用并行实现）中移除所有特殊标记。确保你**分裂**在您的特殊标记上，这样就不会发生跨越它们所分隔文本的合并。例如，如果您有一个语料库（或文本块），例如 [文档 1]<|endoftext|>[文档 2]，你应该在特殊标记上拆分 <|endoftext|>，并预先分词[文档 1]和 [文档 2]分别进行，这样就不会发生跨文档边界的合并。这可以通过以下方式实现：关于。分裂和"`|`"

。join(特殊标记)作为分隔符（谨慎使用）关于。逃脱因为 | 可能出现在特殊标记中）。测试 test_train_bpe_special_tokens我会对此进行测试。

优化合并步骤上述简化示例中 BPE 训练的简单实现速度较慢，因为每次合并操作都需要遍历所有字节对以识别出现频率最高的字节对。然而，每次合并后唯一会改变的字节对计数是与合并后的字节对重叠的字节对。因此，可以通过索引所有字节对的计数并逐步更新这些计数来提高 BPE 训练速度，而不是显式地遍历每个字节对来计算其频率。虽然需要注意的是，这种缓存方法可以显著提高 BPE 训练的速度，但合并操作本身也会影响训练速度。不是Python中可并行化。

资源不足/缩减规模技巧：性能分析

你应该使用分析工具，例如cProfile或者不等边找出实施过程中的瓶颈，并集中精力优化这些瓶颈。

资源不足/缩减规模技巧：“缩减规模”

我们建议您不要直接使用完整的 TinyStories 数据集来训练分词器，而是先使用一小部分数据进行训练，即“调试数据集”。例如，您可以使用 TinyStories 的验证集来训练分词器，该验证集包含 2.2 万个文档，而不是 212 万个文档。这体现了一种尽可能缩小规模以加快开发的通用策略：例如，使用更小的数据集、更小的模型规模等等。选择调试数据集或超参数配置的大小需要仔细考虑：调试数据集要足够大，以便与完整配置具有相同的瓶颈（这样您所做的优化才能具有泛化性），但又不能太大，以免运行时间过长。

问题（train_bpe）：BPE分词器培训（15分）

交付成果编写一个函数，给定一个输入文本文件的路径，训练一个（字节级）BPE 分词器。你的 BPE 训练函数应该（至少）处理以下输入参数：

输入路径：斯特包含 BPE 分词器训练数据的文本文件路径。

词汇量：整数定义最大最终词汇量（包括）的正整数
初始字节词汇表、合并产生的词汇表项以及任何特殊标记）。

特殊标记：列表[斯特]要添加到词汇表中的字符串列表。这些特殊标记不
否则会影响BPE培训。

您的 BPE 训练函数应返回生成的词汇表和合并结果：

词汇：词典[整数，字节]分词器词汇表，一个映射整数（词汇表中的标记 ID
拉里）到字节（标记字节）。

合并：列表[元组[字节，字节]]训练过程中生成的 BPE 合并列表。每个列表项
是一个元组的字节（<token1>，<token2>），表示 <token1>已与 <token2>。合并操作应按创建顺序进行。

要使用我们提供的测试用例来测试您的 BPE 训练函数，您首先需要实现测试适配器。[\[adapters.run_train_bpe\]](#)然后运行uv 运行 pytest tests/test_train_bpe.py。您的实现应该能够通过所有测试。您也可以选择（这可能需要投入大量时间），使用系统语言（例如 C++）来实现训练方法的关键部分（请考虑使用 C++）。cpyyy为此，您可以使用 Python 或 Rust（使用 PyO3）。如果您这样做，请注意哪些操作需要复制，哪些操作需要直接从 Python 内存读取，并确保留下构建说明，或者确保它仅使用 Python 进行构建。pyproject.toml。另请注意，大多数正则表达式引擎对 GPT-2 正则表达式的支持并不完善，即使支持，速度也会很慢。我们已经验证 Oniguruma 的速度相当快，并且支持负向先行断言，但是……正则表达式Python 中的包甚至更快。

问题 (train_bpe_tinystories) : TinyStories 的 BPE 培训 (2 分)

- (a) 使用最大词汇量为 10,000 的 TinyStories 数据集，训练一个字节级 BPE 分词器。确保添加 TinyStories <|文本结束|> 将特殊标记添加到词汇表中。将生成的词汇表序列化并合并到磁盘以供进一步检查。训练耗时多少小时和多少内存？词汇表中最长的标记是多少？它有意义吗？

资源需求： ≤30分钟（无 GPU），≤30GB 内存

暗示使用 BPE 训练，你应该能够在 2 分钟内完成。多进程在预分词过程中，需要考虑以下两个事实：

- (a) <|文本结束|> 标记用于分隔数据文件中的文档。
- (b) <|文本结束|> 在应用 BPE 合并之前，令牌将作为特殊情况进行处理。

交付成果用一到两句话回答。

- (b) 分析你的代码。分词器训练过程中哪个部分耗时最长？ **交付成果**用一到两句话回答。

接下来，我们将尝试在 OpenWebText 数据集上训练一个字节级 BPE 分词器。和之前一样，我们建议您先查看一下该数据集，以便更好地了解其内容。

问题 (train_bpe_expts_owt): BPE OpenWebText 培训 (2 分)

- (a) 使用最大词汇量为 32,000 的 OpenWebText 数据集训练一个字节级 BPE 分词器。将生成的词汇表序列化并合并到磁盘以供进一步检查。词汇表中最长的词元是多少？它有意义吗？

资源需求： ≤12小时（无 GPU），≤100GB 内存 **交付成果**用一到两句话回答。

- (b) 比较 TinyStories 和 OpenWebText 训练得到的分词器，并对比它们之间的区别。 **交付成果**用一到两句话回答。

2.6 BPE分词器：编码和解码

在前一部分作业中，我们实现了一个函数，用于训练一个基于输入文本的 BPE 分词器，从而获得分词器词汇表和 BPE 合并列表。现在，我们将实现一个 BPE 分词器，它加载提供的词汇表和合并列表，并使用它们对文本进行编码和解码，生成相应的词元 ID。

2.6.1 文本编码

使用 BPE 对文本进行编码的过程与我们训练 BPE 词汇表的过程类似。主要包括以下几个步骤。

步骤 1：预分词。我们首先对序列进行预分词，并将每个预分词符表示为 UTF-8 字节序列，就像我们在 BPE 训练中所做的那样。我们将把每个预分词符内的这些字节合并成词汇元素，独立处理每个预分词符（不跨预分词符边界合并）。**步骤 2：应用合并。**然后，我们将 BPE 训练过程中创建的词汇元素合并序列应用于我们的预标记。按照同样的创造顺序。

例子 (bpe_encoding):BPE编码示例

例如，假设我们的输入字符串是“猫吃了我们的词汇是{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}，我们学习到的合并是[(b't', b'h'), (b' ', b'c'), (b' ', 'a'), (b'th', b'e'), (b' a', b't')].首先，我们的预分词器会将此字符串拆分为['这', '猫', '吃了']。然后，我们将查看每个预令牌并应用 BPE 合并。

第一个前缀“这”最初表示为[b't'、b'h'、b'e']。查看我们的合并列表，我们发现第一个适用的合并是（b't', b'h'），并使用它来将前缀转换为[b'th', b'e']。然后，我们返回合并列表，确定下一个适用的合并是（b'th', b'e'），它将前缀转换为[b'the']。最后，回顾合并列表，我们发现没有更多合并操作适用于该字符串（因为整个前缀标记已合并为一个标记），因此我们已完成 BPE 合并。对应的整数序列为 [9]。

对剩余的前缀重复此过程，我们发现前缀“猫”表示为 [b' c', b'a', b't']应用 BPE 合并后，得到整数序列 [7、1、5]。最后一个前缀“吃”是 [b' at', b'e']应用 BPE 合并后，得到整数序列 [10, 3]。因此，对输入字符串进行编码的最终结果是[9、7、1、5、10、3]。

特殊代币。您的分词器应该能够正确处理用户定义的特殊标记，以便在对文本进行编码时使用（在构建分词器时提供）。

内存方面的考虑。假设我们要对一个无法完全加载到内存中的大型文本文件进行词法分析。为了高效地对这个大型文件（或其他任何数据流）进行词法分析，我们需要将其分割成易于管理的块，并依次处理每个块，从而使内存复杂度保持恒定，而不是与文本大小呈线性关系。在此过程中，我们需要确保词法单元不会跨越块边界，否则我们将得到与直接在内存中对整个序列进行词法分析的简单方法不同的结果。

2.6.2 解码文本

要将一系列整数标记 ID 解码回原始文本，我们可以简单地在词汇表中查找每个 ID 对应的条目（一个字节序列），将它们连接起来，然后将这些字节解码为 Unicode 字符串。请注意，输入的 ID 不一定能映射到有效的 Unicode 字符串（因为用户可以输入任意整数 ID 序列）。如果输入的标记 ID 无法生成有效的 Unicode 字符串，则应将格式错误的字节替换为官方的 Unicode 替换字符。U+FFFD。³这错误论证字节解码控制如何处理 Unicode 解码错误，并使用errors='替换'将自动用替换标记替换格式错误的格式错误的字符串。

问题（分词器）：实现分词器（15分）

交付成果实施分词器该类接受一个词汇表和一个合并列表，并将文本编码为整数 ID，再将整数 ID 解码为文本。您的分词器还应支持用户提供的特殊标记（如果词汇表中尚不存在，则将其添加到词汇表中）。我们推荐以下接口：

定义__init__（自己词汇表、合并、特殊标记=没有任何）根据给定的数据构建分词器
词汇表、合并列表以及（可选的）特殊标记列表。此函数应接受

³有关 Unicode 替换字符的更多信息，请参阅 [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

以下参数：

词汇：词典[整数，字节合并：列表[元组[字节，字节]] 特殊标记：列表[斯特]|没有任何=没有任何

定义来自文件 (`cls`, `vocab_filepath`, `merges_filepath`, `special_tokens=没有任何`) 班级
构造并返回一个的方法分词器该方法从序列化的词汇表和合并列表（格式与您的 BPE 训练代码输出格式相同）以及（可选的）特殊标记列表中提取信息。此方法还应接受以下附加参数：

词汇文件路径：斯特

`merges_filepath`：斯特

特殊标记：列表[斯特]|没有任何=没有任何

定义编码 (`自己`, `文本`：斯特) ->列表[整数]将输入文本编码成一系列标记 ID。

定义可迭代编码 (`自己`, `可迭代对象`：可迭代对象[斯特]) ->迭代器[整数]给定一个可迭代对象字符串（例如 Python 文件句柄）返回一个生成器，该生成器会延迟生成标记 ID。这对于高效地对无法直接加载到内存中的大型文件进行标记化是必需的。

定义解码 (`自己ids`：列表[整数]) ->斯特将一系列令牌 ID 解码为文本。

为了测试你的分词器针对我们提供的测试，您首先需要实现测试适配器。`[adapters.get_tokenizer]`然后运行uv 运行 pytest tests/test_tokenizer.py。你的实现应该能够通过所有测试。

2.7 实验

问题 (tokenizer_experiments):分词器实验 (4 分)

- (a) 从 TinyStories 和 OpenWebText 中分别抽取 10 个文档样本。使用你之前训练的 TinyStories 和 OpenWebText 分词器（词汇量分别为 10K 和 32K），将这些抽取的文档编码为整数 ID。每个分词器的压缩比（字节/词元）是多少？

交付成果用一到两句话回答。

- (b) 如果使用 TinyStories 分词器对 OpenWebText 样本进行分词，会发生什么？比较压缩率和/或定性描述所发生的情况。

交付成果用一到两句话回答。

- (c) 估算你的分词器的吞吐量（例如，以字节/秒为单位）。对 Pile 数据集（825GB 文本）进行分词需要多长时间？

交付成果用一到两句话回答。

- (d) 使用 TinyStories 和 OpenWebText 分词器，将各自的训练数据集和开发数据集编码成一系列整数标记 ID。我们稍后将使用这些 ID 来训练语言模型。我们建议将标记 ID 序列化为 NumPy 数据类型的数组。uint16。为什么是 uint16 合适的选择吗？

交付成果用一到两句话回答。

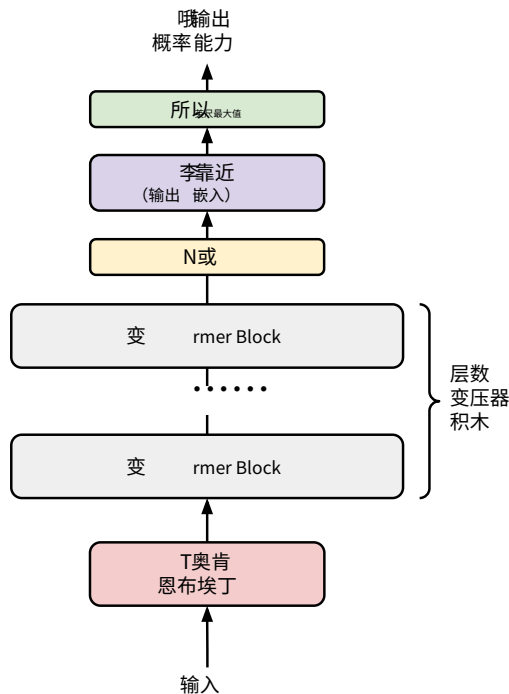


图 1：我们的 Transformer 语言模型概述。

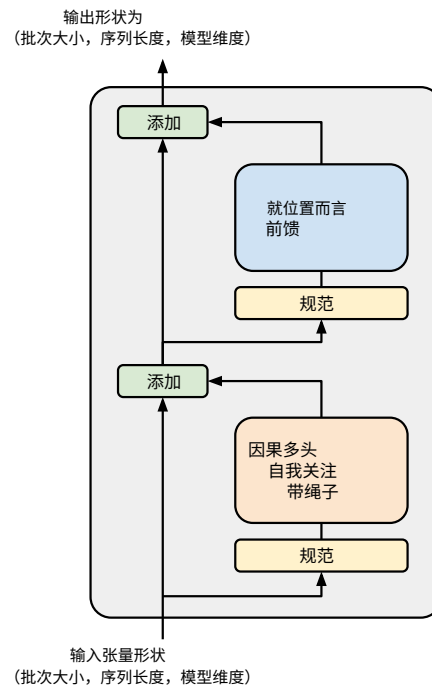


图 2：预范数 Transformer 模块。

3 变 rmer 语言 e 模型架构

一语言模块 el 以 ba 作为输入 序列 已处理的整数标记 ID 序列 (IE, 火炬。张量形状概率分 e (batch_size, se 长度)), 一个 d 返回一个 (批量) 标准化的 pro hape 布 th, vocab_size)), 在哪里 e vo 卡布拉里 (即, 一 PyTorch s 的张量 (批次大小, 序列长度) e 预测分布 接下来的一段时间是…… 西对每个输入进行排序 肯。在训练语言模型时, w e 使用这些下一个 wo rd 预测计算 计算交叉熵 实际损失 下一个词和 th 接受 e 预测下一个世界 rd. 当从最终的 ti 生成 t 在推理过程中, 我们从语言模型中提取信息, 然后 预测 d 下一个词分发 tion 时, 例如, 通过 从序列中采样, 并重复执行, 直到找到概率最高的对下一次代币分配 在序列中 (e 添加 toke token 带到输入 n 基因 (即序列中的最后一个项目)。 进行评分等)。

生成的 se 赋值, 您

在这一部分 我们将从以下方面构建这个 Transformer 语言模型 m scratch。我们将 以高开头 - 级别描述 在逐步详细描述个体之前, 先建立模型 双组分。

3.1 传输 mer LM

给定一个序列 令牌 ID, Tra nsformer 语言模型使用输入嵌入来转换词元 将 ID 转换为密集向量, 并将嵌入的标记传递出去层数 Transformer 模块随后应用学习到的线性投影 (“输出嵌入” 或 “LM 头”) 来生成预测的下一个词元。 logits。示意图见图 1。

3.1.1 词嵌入

第一步, 变形金刚嵌入式将 (批量) 标记 ID 序列转换为包含标记身份信息的向量序列 (图 1 中的红色方块)。

更具体地说，给定一个词元 ID 序列，Transformer 语言模型使用词元嵌入层生成一个向量序列。每个嵌入层接收一个形状为 (批次大小, 序列长度) 并生成一系列形状为 (batch_size、sequence_length、d_model)。

3.1.2 预规范转换器模块

嵌入之后，激活值由几个结构相同的神经网络层进行处理。一个标准的仅解码器Transformer语言模型包含：层数相同的图层（通常称为 Transformer “块”）。每个 Transformer 块接收一个形状输入 (batch_size、sequence_length、d_model) 并返回形状为 (batch_size、sequence_length、d_model)。每个模块聚合序列中的信息（通过自注意力机制），并对其非线性变换（通过前馈层）。

3.2 输出归一化和嵌入

后层数Transformer 模块，我们将获取最终激活值，并将其转换为词汇表上的分布。

我们将实现“预归一化”Transformer模块（详见第3.5节），该模块还需要在最后一个Transformer模块之后使用层归一化（详见下文），以确保其输出得到正确的缩放。

经过归一化之后，我们将使用标准的学习线性变换将 Transformer 模块的输出转换为预测的下一个标记 logits（例如，参见 Radford 等人 [2018] 公式 2）。

3.3 备注：批处理、Einsum 和高效计算

在整个 Transformer 模型中，我们将对多个批次式输入执行相同的计算。以下是一些示例：

- **批次元素**我们应用相同的Transformer向前对每个批次元素进行操作。
- **序列长度**：像 RMSNorm 和前馈这样的“按位置”操作，对序列的每个位置都以相同的方式进行操作。
- **注意头部**注意力操作在“多头”注意力操作中跨注意力头进行批量处理。

以符合人体工程学的方式执行此类操作非常有用，这种方式能够充分利用 GPU，并且易于阅读和理解。许多 PyTorch 操作可以在张量的开头接收额外的“批处理”维度，并高效地在这些维度上重复/广播操作。

例如，假设我们正在执行按位置分批处理的操作。我们有一个“数据张量”。*D*形状（批次大小，序列长度，d_model），我们想对一个矩阵进行批量向量矩阵乘法运算。一个形状 (d_model, d_model)。在这种情况下，*D@*一个将执行批量矩阵乘法，这是 PyTorch 中的一种高效原语，其中（批次大小，序列长度）尺寸是批量处理的。

因此，假设你的函数可能会被赋予额外的批次式维度，并将这些维度保留在 PyTorch 形状的开头，会很有帮助。为了组织张量以便它们能够以这种方式进行批处理，它们可能需要经过多个步骤的形状调整。视图，重塑和转置。这可能有点麻烦，而且通常很难读懂代码在做什么以及张量的形状是什么。

更符合人体工程学的选择是使用 *等和符号* 之内 `torch.einsum`，或者更确切地说，使用与框架无关的库，例如艾诺普斯或者 `einx`。两项关键行动是：简而言之，它可以对任意维度的输入张量进行张量收缩，并且重新排列，它可以对任意字符串进行重新排序、连接和拆分。

维度。事实证明，机器学习中的几乎所有操作都是维度转换和张量收缩的某种组合，偶尔还会用到（通常是逐点的）非线性函数。这意味着，使用 einsum 符号可以使你的代码更易读、更灵活。

我们强烈建议课堂上学习和使用 einsum 符号。未毕业的学生
以前接触过 einsum 符号的人应该使用艾诺普斯（文档在此），以及已经熟悉的学生艾诺普斯应该学习更一般的知识 einx（这里）。⁴我们提供的环境已经安装了这两个软件包。

这里我们提供一些如何使用 einsum 表示法的示例。这些示例是对 einops 文档的补充，您应该先阅读 einops 文档。

例子（爱因斯坦示例1）：批量矩阵乘法 einops.einsum

进口火炬

从艾诺普斯进口重新排列，总和

基本实现是=D@一个。T

很难分辨输入和输出的形状及其含义。

D 和 A 可以有哪些形状？这些形状中是否存在意想不到的行为？

Einsum 具有自文档化和健壮性

D 一个 -> 是
是=einsum(D, A, "批处理序列 d_in, d_out d_in -> 批处理序列 d_out")

或者，批量版本中 D 可以有任意主导维度，但 A 受到约束。是=einsum(D, A, "... d_in, d_out d_in -> ... d_out")

例子（爱因斯坦示例2）：广播操作 einops.重新排列

我们有一批图像，对于每张图像，我们希望根据某个缩放因子生成 10 个变暗版本：

图片=火炬。randn(64, 128, 128, 3) # (批次、高度、宽度、通道) dim_by=火炬。

linspace(开始=0.0, 结尾=1.0步骤=10)

重塑和倍增 维度值=重新排列

(dim_by, "dim_value -> 1 dim_value 1 1")
图像_后部=重新排列 (图像, "b 高度 宽度 通道 -> b 1 高度 宽度 通道") 暗淡的图像=图像_后部*维度值

或者一次性完成：暗淡的图

像=总计

图像, dim_by,
"批处理高度宽度通道, dim_value -> 批处理 dim_value 高度宽度通道"
)

⁴值得注意的是，虽然艾诺普斯获得了大量支持，因克斯尚未经过实战检验。您可以随时使用其他方法。艾诺普斯如果发现任何限制或错误，可以使用更纯粹的 PyTorch 版本进行测试。einx。

例子（爱因斯坦示例3）：像素混合einops.重新排列

假设我们有一批图像，表示为形状为（批次、高度、宽度、通道），我们对图像的所有像素进行线性变换，但每个通道的变换应该独立进行。我们的线性变换可以用一个矩阵表示。 B 形状（高度 $\times$ 宽度，高度 $\times$ 宽度）。

```
channels_last=火炬.randn(64, 32, 32, 3) # (批次、高度、宽度、通道) B=火炬.randn(32*32, 32*32)
```

```
# 重新排列图像张量，以便跨所有像素进行混合 channels_last_flat=
channels_last.看法 (
    -1, channels_last.尺寸 (1) *channels_last.尺寸 (2), channels_last.尺寸 (3)
)
channels_first_flat=channels_last_flat.转置(1, 2)
```

```
channels_first_flat_transformed=channels_first_flat@B. T
```

```
channels_last_flat_transformed=channels_first_flat_transformed.转置(1, 2)
```

```
channels_last_transformed=channels_last_flat_transformed.看法 (*channels_last.形状)
```

相反，使用einops：

```
高度=宽度= 32
```

```
## 重新排列取代笨拙的火炬视图 + 转置 channels_first=重新排列 (
```

```
    channels_last,
    "批次高度宽度通道 -> 批次通道 (高度宽度)"
)
```

```
通道首先转换=总计
```

```
    channels_first, B,
    "批处理通道 pixel_in, pixel_out pixel_in -> 批处理通道 pixel_out"
)
```

```
channels_last_transformed=重新排列 (
```

```
    channels_first_transformed,
    "批处理通道 (高度宽度) -> 批处理高度宽度通道"， 高度=高度， 宽度=宽度
)
```

或者，如果你感觉自己疯了：一次性全部用完因克斯。点（einx等价物艾诺普斯。总和）

```
高度=宽度= 32 channels_last_transformed=
```

```
因克斯。点 (
```

```
    "批处理 row_in col_in 通道， (row_out col_out) (row_in col_in) " "-> 批
    处理 row_out col_out 通道"， channels_last, B,
```

```
    col_in=宽度, col_out=宽度
)
```

这里的第一个实现可以通过在前后添加注释来改进，以表明……

输入和输出的形状是什么，但这很笨拙且容易出错。使用 einsum 表示法，文档会更清晰。是执行！

Einsum 表示法可以处理任意输入批次维度，但它还有一个关键优势：*自我记录*使用 einsum 表示法的代码可以更清晰地展现输入和输出张量的相关形状。对于其余的张量，您可以考虑使用张量类型提示，例如使用……jaxtyping库（并非 Jax 特有）。

我们将在作业 2 中更详细地讨论使用 einsum 符号的性能影响，但现在只需知道它们几乎总是比其他方法更好！

3.3.1 数学符号和内存排序

许多机器学习论文在其表示法中使用行向量，这使得它们的表示方式与 NumPy 和 PyTorch 默认使用的行优先内存顺序非常契合。使用行向量，线性变换看起来像这样

$$y = xW_T, \quad (1)$$

行主 $W \in \text{拉}_{d_{\text{出去}} \times d_{\text{在}}}$ 和行向量 $x \in \text{拉}_{1 \times d_{\text{在}}}$ 。

在线性代数中，通常更常用列向量，而线性变换则表现为……
喜欢

$$y = Wx, \quad (2)$$

给定一个行主序 $W \in \text{拉}_{d_{\text{出去}} \times d_{\text{在}}}$ 和列向量 $x \in \text{拉}_{d_{\text{在}}}$ 。我们将使用列向量。为了便于理解，本作业中将使用数学符号。请注意，如果您想使用普通的矩阵乘法符号，则必须按照行向量约定来处理矩阵，因为 PyTorch 使用行优先的内存顺序。总计对于你的矩阵运算来说，这应该不是问题。

3.4 基本构建模块：线性模块和嵌入模块

3.4.1 参数初始化

有效训练神经网络通常需要仔细初始化模型参数——错误的初始化会导致梯度消失或梯度爆炸等不良行为。预范数 Transformer 对初始化具有异常强的鲁棒性，但初始化仍然会对训练速度和收敛性产生显著影响。由于本次作业篇幅较长，我们将把细节留到作业 3 中讲解，这里提供一些适用于大多数情况的近似初始化方法。目前，请使用：

- 线性权重： $N(\mu=0, \sigma^2 = \frac{2}{d_{\text{在}} + d_{\text{出去}}})$ 截断于 $[-3\sigma, 3\sigma]$
- 嵌入： $N(\mu=0, \sigma^2=1)$ 截断于 $[-3, 3]$
- RMSNorm： 1

你应该使用火炬.nn。初始化。截断正态初始化截断正态权重。

3.4.2 线性模块

线性层是 Transformer 模型和神经网络的基本组成部分。首先，你需要实现自己的线性层。线性继承自 torch.nn. 模块并执行线性变换：

$$y = Wx + b. \quad (3)$$

请注意，我们没有包含偏差项，这与大多数现代线性模型一致。

问题（线性）：实现线性模块（1分）

交付成果 实现线性继承自 `torch.nn.Module` 并执行线性变换。你的实现应该遵循 PyTorch 内置函数的接口。`nn.Linear` 模块，除了没有一个偏见参数或实参。我们推荐以下接口：

定义 `__init__`（自己，输入功能，输出功能，设备=没有任何，dtype=没有任何）构建一个线性变换模块。此函数应接受以下参数：

in_features: 整数 输入的最终维度 **输出特征:** 整数

输出的最终尺寸

设备: 手电筒。设备|没有任何=没有任何用于存储参数的设备 **数据类型:** torch

。数据类型|没有任何=没有任何参数的数据类型

定义向前（自己，x: 火炬。张量）->火炬。张量 对 进行线性变换输入。

务必：

- 子类 `nn.Module`
- 调用超类构造函数
- 构建并存储您的参数，作为 W （不是 W^T 出于内存排序的考虑，将其放入一个 `nn.Parameter` 范围

当然，不要使用 `nn.Linear` 或者 `nn.Linear` 的功能性。线性

对于初始化，请使用上述设置以及火炬。`nn.Linear`。初始化。截断正态初始化权重。

为了测试你的线性模块中，实现测试适配器 `[adapters.run_linear]` 适配器应将给定的权重加载到您的设备中。线性模块。您可以使用模块。加载状态字典为此目的，然后运行 `uv run pytest -k test_linear`。

3.4.3 嵌入模块

如上所述，Transformer 的第一层是嵌入层，它将整数标记 ID 映射到一个维度为 d_{model} 的向量空间。`d_model`。我们将实施一项定制方案嵌入继承自 `torch.nn.Module`（所以你不应该使用 `nn.Embedding`）。这向前该方法应通过索引到形状为 $(\text{vocab_size}, d_{\text{model}})$ 的嵌入矩阵中，为每个标记 ID 选择嵌入向量。`vocab_size, d_model` 使用 `torch.LongTensor` 形状为 $(\text{batch_size}, \text{seq_length})$ 的标记 ID 批次大小，序列长度）。

问题（嵌入）：实现嵌入模块（1分）

交付成果 实现嵌入继承自 `torch.nn.Module` 并执行嵌入查找。您的实现应遵循 PyTorch 内置函数的接口。`nn.Embedding` 模块。我们推荐以下接口：

定义 `__init__`（自己，num_embeddings, embedding_dim, 设备=没有任何，dtype=没有任何）构造一个嵌入模块。该函数应接受以下参数：

num_embeddings: 整数 词汇量

嵌入维度：整数嵌入向量的维度，即 $d_{\text{模型}}$

设备：手电筒。设备|没有任何=没有任何用于存储参数的设备 **数据类型：** torch

。数据类型|没有任何=没有任何参数的数据类型

定义向前（自己token_ids: torch。张量）->火炬。张量查找嵌入向量

对于给定的令牌 ID。

务必：

- 子类nn。模块
- 调用超类构造函数
- 将嵌入矩阵初始化为nn。范围
- 将嵌入矩阵存储起来d_model成为最终维度

当然，不要使用nn。嵌入或者nn。功能性。嵌入

同样，使用上述设置进行初始化，并使用火炬。nn。初始化。截断正态初始化权重。

要测试您的实现，请在以下位置实现测试适配器：[\[adapters.run_embedding\]](#)然后运行 uv 运行 pytest -k test_embedding。

3.5 预规范变压器模块

每个 Transformer 模块有两个子层：多头自注意力机制和位置前馈网络（Vaswani 等人，2017 年，第 3.1 节）。

在最初的Transformer论文中，模型在每个子层周围使用残差连接，然后进行层归一化。这种架构通常被称为“后归一化”Transformer，因为层归一化应用于子层的输出。然而，大量研究表明，将层归一化从每个子层的输出移到每个子层的输入（并在最后一个Transformer模块之后进行额外的层归一化）可以提高Transformer的训练稳定性[Nguyen和Salazar, 2019; Xiong等人, 2020]——图2展示了这种“前归一化”Transformer模块的可视化表示。然后，每个Transformer模块子层的输出通过残差连接添加到子层的输入中（Vaswani等人，2017，第5.4节）。预归一化的一个直观理解是，它提供了一个干净的“残差流”，从输入嵌入到Transformer的最终输出之间没有任何归一化，据称这可以改善梯度流。这种预归一化Transformer现在是语言模型（例如 GPT-3、LLaMA、PaLM等）的标准，因此我们将实现这个变体。我们将逐步介绍预归一化Transformer模块的每个组成部分，并按顺序实现它们。

3.5.1 均方根层归一化

Vaswani 等人 [2017] 的原始 Transformer 实现使用层归一化 [Ba 等人，2016] 对激活值进行归一化。根据 Touvron 等人 [2023] 的做法，我们将使用均方根层归一化（RMSNorm；Zhang 和 Sennrich，2019，公式 4）进行层归一化。给定一个向量 $\text{一个} \in \mathbb{R}^{d_{\text{模型}}}$ 对于每个激活值，RMSNorm 将重新缩放每个激活值。一个 我 具体如下：

$$\text{RMSNorm}(\text{一个}) = \frac{\text{一个} \text{我}}{\text{RMS}(\text{一个})} \quad (4)$$

其中 $\text{RMS}(\text{一个}) = \sqrt{\frac{1}{d_{\text{模型}}} \sum_{\text{我}=1}^{d_{\text{模型}}} \text{一个}^2_{\text{我}}}$ 。这里， 我 是一个可学习的“增益”参数（有 $d_{\text{模型}}$ 这样的参数总数），以及 ϵ 是一个超参数，通常固定为 $1e-5$ 。

你应该将输入向上转换为火炬。float32为了防止输入平方时发生溢出。总而言之，您的向前方法应如下所示：

```
数据类型=x.dtype x=x.to(
    (火炬)。float32)
```

```
# 此处编写执行 RMSNorm 的代码
```

```
.....
```

```
结果= ...
```

```
# 以原始数据类型返回结果 返回结果。到
    (in_dtype)
```

问题（rmsnorm）：均方根层归一化（1分）

交付成果：实现 RMSNorm 作为torch.nn.模块。我们推荐以下界面：

定义__init__（自己d_model：整数，eps：漂浮= 1e-5，设备=没有任何，dtype=没有任何）

构建 RMSNorm 模块。该函数应接受以下参数：

d_model：整数模型的隐藏维度 **EPS：**漂浮= 1e-5数值稳定性

性ε值

设备：手电筒。设备|没有任何=没有任何用于存储参数的设备 **数据类型：**torch

。数据类型|没有任何=没有任何参数的数据类型

定义向前（自己，x：火炬。张量）->火炬。张量处理形状为

（批次大小，序列长度，d_model）并返回相同形状的张量。

笔记：请记得将您的输入向上转发。torch.float32在执行规范化（以及之后向下转型为原始数据类型）之前，如上所述。

要测试您的实现，请在以下位置实现测试适配器：[\[adapters.run_rmsnorm\]](#)然后运行uv 运行 pytest -k test_rmsnorm。

3.5.2 基于位置的前馈网络

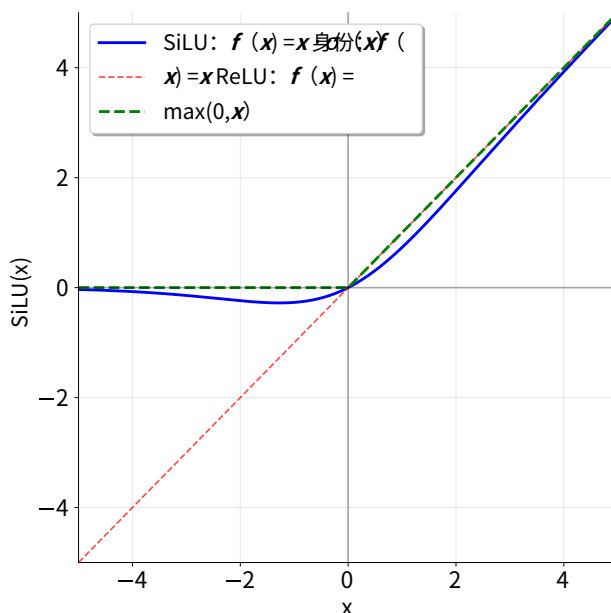


图 3: Comparing SiLU (又名 Swish) 和 ReLU 活动变化功能。

在原产地最终变压器 paper (Vaswani 的第 3.3 节, 使用 ReLU) 激活 (ReLU (x) = max(0, x)) 之间。它们由……组成 f 两个线性变换 尺寸 在内在感受的本质, 现代前向层是典型的模型, 输入调光速率提高 4 倍, 主要变化机制有两倍, 主要变化机制有两倍, 主要变化机制有两倍。我们将……最终设计: 他们使用其他激活功能 n 一个 LM 那样采用 羊驼 3 [Grattafiori et al., 2024] 和 Qwen 2.5 [杨 s 我被称为 o 最翼 “SwiGLU” 激活函数 一个 LM 那样采用 d Swish) 激活 w s 有时使用的术语 和 LLaMA [Touvron 2023]。E [Lfwing 等人, 2017] 定义为等, 2024], 它结合了耳部单元 SiLU (通常也省略了 封闭式 L 我 (GLU)。我们 LMs, bia wdherly 等人, 现代的 L 因为 PaLM [Chen 或 o2022] n 函数 o [Hendryck s 和 Gimpel, 2016 年 E [Lfwing 等人, 2017] i s 定义为这 S 我 wish 激活 o [Hendryck s 和 Gimpel, 2016 年 E [Lfwing 等人, 2017] i s 定义为

接下来是:

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

可以 如图 3 所示, 零点处 SiLU 激活 f 你 action 与 t 类似 he ReLU 激活函数 nction, 但是光滑 可见。封闭式线性单元 (GLU) 最初定义 Dauphin 等人 [2017] 作为元素-线性变 使用产品线性 转换通道 ed 通过 S 形曲线 f 和另一个 我:

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

在哪里 $\odot$ 代表元素-wis e 乘法。Gat ed 线性单元是 gr 的线 建议 “在保留……的同消失坡度 p 深层建筑的问题。” t 通过提供讲座 一性路径。一时减少……非线性能力 e 小天使用 SiLU/Swish and GLU 一起, 我们 get SwiGLU, 其中 h 我们将用于我们的 f 向前网络:

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

在哪里 $x \in \mathbb{R}^{d_{\text{ff}}}$ 拉 d 模型, $W_1, W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{ff}}}$ 拉 d 模型, $W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{ff}}}$ 拉 d 模型, 并且按正典来说, $d_{\text{ff}} = 8 \times d_{\text{ff}} / 3$ 模型。

Shazeer [2020] 首先提出将 SiLU/Swish 激活函数与 GLU 函数相结合，并通过实验证明，在语言建模任务上，SwiGLU 的性能优于 ReLU 和 SiLU（无门控）等基线方法。在后续的作业中，您将比较 SwiGLU 和 SiLU。尽管我们已经提到了一些关于这些组件的启发式论证（论文也提供了更多佐证），但保持实证视角仍然很重要：Shazeer 论文中有一句著名的引言是：

我们不解释为什么这些架构似乎有效；我们将它们的成功，如同其他一切一样，归功于神的恩惠。

问题（位置前馈）：实现位置前馈网络（2 分）

交付成果实现由 SiLU 激活函数和 GLU 组成的 SwiGLU 前馈网络。

笔记：在这种情况下，您可以随意使用火炬。乙状结肠在你的数值稳定性实现中。

你应该设置 d_{ff} 大约 $\frac{2}{3} \times d_{\text{模型}}$ 在您的实施过程中，同时确保为了充分利用硬件，内部前馈层的维度应为 64 的倍数。要使用我们提供的测试用例来测试您的实现，您需要实现测试适配器。[adapters.run_swiglu] 然后运行 `uv run pytest -k test_swiglu` 测试你的实现。

3.5.3 相对位置嵌入

为了将位置信息注入模型，我们将实现旋转位置嵌入（Rotary Position Embeddings, RoPE）[Su et al., 2021]。对于给定的查询词元，我们将采用 RoPE 方法。 $q_{(i)} = W_q x_{(i)} \in \mathbb{R}^{d_q}$ 在标记位置 i 我们将应用成对旋转矩阵 $R_{i,k}$ 给我们 $q'_{(i)} = R_{i,k} q_{(i)} = R_{i,k} W_q x_{(i)}$ 。这里， $R_{i,k}$ 将旋转嵌入对

元素 $q_{(i)}[2k-1:2k]$ 以角度表示为二维向量 $\theta_{i,k} = \frac{\Theta}{(2) \lfloor \frac{d}{2} \rfloor} i$ 为了 $k \in \{1, \dots, d/2\}$ 以及某个常数 Θ 。因此，我们可以考虑 $R_{i,k}$ 是一个大小为 $d \times d$ 用方块 $R_{i,k}$ 为了 $k \in \{1, \dots, d/2\}$ ，和

$$R_{i,k} = \begin{bmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{bmatrix}. \quad (8)$$

由此我们得到完整的旋转矩阵。

$$R_i = \begin{bmatrix} R_{i,1} & 0 & 0 & \dots & 0 \\ 0 & R_{i,2} & 0 & \dots & 0 \\ 0 & 0 & R_{i,3} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & R_{i,d/2} \end{bmatrix}, \quad (9)$$

在哪里 0_s 代表 2×2 零矩阵。虽然可以构造完整的 $d \times d$ 对于矩阵，一个好的解决方案应该利用该矩阵的性质来更高效地实现变换。由于我们只关心给定序列中标记的相对旋转，我们可以重用我们计算出的 $\cos(\theta_{i,k})$ 和 $\sin(\theta_{i,k})$ 跨层和不同批次。如果想要优化，可以使用所有层引用的单个 RoPE 模块，该模块可以在初始化期间创建一个预先计算的二维正弦和余弦值缓冲区。自己。注册缓冲区（持久性）=错误的，而不是一个 nn。范围（因为我们不想学习这些固定的余弦和正弦值）。我们对……也进行了完全相同的旋转过程。 $q_{(i)}$ 那就完成了。 $k \in \{1, \dots, d/2\}$ 旋转相应的角度 $\theta_{i,k}$ 请注意，这一层没有可学习的参数。

问题（绳索）：实施绳索（2分）

交付成果实现一个类旋转位置嵌入将 RoPE 应用于输入张量。推荐使用以下接口：

定义__init__（自己， θ ：漂浮， d_k ：整数，max_seq_len：整数，设备=没有任何）构造使用 RoPE 模块，并根据需要创建缓冲区。

θ ：漂浮 θ 绳索的价值 d_k ：整数查询维度和键向量

max_seq_len：整数输入序列的最大长度 设备：手电筒。设备|没有任何=没有任何用于存储缓冲区的设备

定义向前（自己，x：火炬。张量，token_positions：torch。张量）->火炬。张量
处理形状为（...，seq_len， d_k ）并返回一个相同形状的张量。请注意，您应该容忍这种情况。x具有任意数量的批次维度。您应该假设标记位置是一个形状为（...，seq_len）指定标记位置x沿序列维度。

你应该使用标记位置来沿序列维度对（可能预先计算的）余弦和正弦张量进行切片。

要测试您的实现，请完成[adapters.run_rope]并确保它通过。uv 运行 pytest -k test_rope。

3.5.4 缩放点积注意力

现在我们将实现 Vaswani 等人 [2017]（第 3.2.1 节）中描述的缩放点积注意力机制。作为初步步骤，注意力操作的定义将使用 softmax 函数，该函数接受一个未归一化的分数向量，并将其转换为归一化分布：

$$\text{softmax}(v) = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)} \quad (10)$$

注意 $\exp(v_i)$ 可以成为信息对于较大的值（那么， $\inf/\inf = \text{NaN}$ ）。我们可以通过注意到 softmax 运算对于加上任何常数都是不变的来避免这种情况。c 对所有输入。我们可以利用这一特性来保证数值稳定性——通常，我们会减去最大的输入值。o 我来自所有元素 o 我成为新的最大参赛者 0。现在您将实现 softmax 函数，并使用此技巧来保证数值稳定性。

问题（softmax）：实现 softmax 函数（1分）

交付成果编写一个函数，对张量应用 softmax 操作。该函数应接受两个参数：一个张量和一个方面我并对……应用 softmax 我输入张量的第 n 维。输出张量应与输入张量具有相同的形状，但其我第 i 个维度现在将具有归一化的概率分布。使用从第 i 个维度中减去最大值的技巧。我第-维，来自所有元素我为了避免数值稳定性问题，需要考虑第-维维度。

要测试您的实现，请完成[adapters.run_softmax]并确保它通过。uv 运行 pytest -k test_softmax_matches_pytorch。

现在我们可以用数学方法将注意力操作定义如下：

$$\text{注意力}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{d_k}\right)V \quad (11)$$

在哪里 $Q \in \text{拉}_{n \times d_k}$, $K \in \text{拉}_{m \times d_k}$, 和 $V \in \text{拉}_{m \times d_v}$ 。这里, Q , K 和 V 这些都是此操作的输入——请注意, 这些不是可学习的参数。如果您想知道为什么这不是 QK^T 参见 3.3.1。掩蔽: 有时这样做很方便 面具 注意力机制的输出。掩码应具有以下形状 $M \in \{\text{真的}, \text{错误的}\}_{n \times m}$ 以及每一行 i 该布尔矩阵指示查询的键 j 我应该注意。通常 (而且有点令人困惑), 一个值真的在位置 (i, j) 这表明查询 i 做注意关键 j , 以及一个值错误的这表明查询 i 不注意关键点。换句话说, “信息流” 位于 (i, j) 有值真的。例如, 考虑以下情况: 1×3 掩码矩阵, 其元素为 $[[\text{真}, \text{真}, \text{假}]]$ 。单个查询向量只关注前两个键。

从计算角度来看, 使用 m(asking) 比计算后续事件的注意力要高效得多。序列, 我们可以通过获取预softmax值来实现这一点。 $\frac{QK^T}{d_k}$ 并添加一个 $-\infty$ 在任何条目中掩码矩阵为 False。

问题 (scaled_dot_product_attention): 实现缩放点积注意力机制 (5 分)

交付成果 实现缩放点积注意力函数。你的实现应该能够处理形状为 $(\text{batch_size}, \dots, \text{seq_len}, d_k)$ 以及形状的价值 $(\text{batch_size}, \dots, \text{seq_len}, d_v)$, 其中 $\dots$ 表示任意数量的其他批次状维度 (如果提供)。实现应返回形状为 $(\text{batch_size}, \dots, d_v)$ 。有关批次式维度的讨论, 请参见第 3.3 节。

您的实现还应支持可选的用户提供的布尔掩码, 形状为 $(\text{seq_len}, \text{seq_len})$ 。掩码值为 1 的位置的注意力概率真的所有这些加起来应该等于 1, 并且掩码值为 0 的位置的注意力概率为错误的应该为零。

要使用我们提供的测试用例来测试您的实现, 您需要实现测试适配器。[[adapters.run_scaled_dot_product_attention](#)]

`uv run pytest -k test_scaled_dot_product_attention` 测试你的实现对三阶输入张量的处理能力, 同时 `uv run pytest -k test_4d_scaled_dot_product_attention` 测试你的实现对四阶输入张量的处理能力。

3.5.5 因果多头自注意力

我们将实现 Vaswani 等人 [2017] 的 3.2.2 节中描述的多头自注意力机制。回顾一下, 从数学角度来看, 应用多头注意力机制的操作定义如下:

$$\text{多头}(Q, K, V) = \text{连接}(\text{头部}_1, \dots, \text{头部}_h) \quad (12)$$

$$\text{头部}_i = \text{注意力}(Q_i, K_i, V_i) \quad (13)$$

和 Q_i, K_i, V_i 切片编号 $i \in \{1, \dots, h\}$ 尺寸 d_k 或者 d_v 嵌入维度 Q, K 和 V 分别地, 其中 Attention 是 § 3.5.4 中定义的缩放点积注意力操作。由此我们可以构建多头模型。自己-注意操作:

$$\text{多头自注意力机制}(x) = W_{\text{多头}} \text{多头}(W_Q x, W_K x, W_V x) \quad (14)$$

这里, 可学习参数是 $W_Q \in \text{拉}_{\text{拉}_{\text{高清}} \times d_{\text{模型}}}$, $W_K \in \text{拉}_{\text{拉}_{\text{高清}} \times d_{\text{模型}}}$, $W_V \in \text{拉}_{\text{拉}_{\text{高清}} \times d_{\text{模型}}}$, 和 $W_{\text{多头}} \in \text{拉}_{d_{\text{模型}} \times \text{拉}_{\text{高清}}}$ 。自 Q, K , 和 V 在多头注意力操作中, s 被切片, 我们可以这样想: W_Q, W_K 和 W_V 沿着输出维度, 每个头部都会被分开。完成此操作后, 您应该只需三次矩阵乘法即可计算出键、值和查询投影。⁵

⁵作为一项挑战目标, 尝试将键、查询和值投影合并到一个权重矩阵中, 这样就只需要一次矩阵乘法运算。

因果掩蔽。你的实现应该阻止模型关注序列中的后续标记。换句话说，如果模型被赋予一个标记序列，那么模型就不应该再关注后续标记。 $t_1, \dots, t_n$ 我们希望计算前缀的下一个词的预测结果。 $t_1, \dots, t_i$ （在哪里 $i < n$ ），该模型应该不是能够访问（处理）位置处的令牌表示 $t_{i+1}, \dots, t_n$ 因为在推理过程中生成文本时，它将无法访问这些标记（而且这些未来的标记会泄露关于下一个真实单词的信息，从而使语言模型预训练的目标变得无关紧要）。对于输入标记序列 $t_1, \dots, t_n$

我们可以通过运行多头自注意力机制来简单地阻止对未来代币的访问。 n （对于 n 序列中的唯一前缀）。相反，我们将使用因果注意力掩蔽，它允许标记 i 负责所有岗位 $j \leq i$ 按顺序使用。torch.triu 或者使用广播索引比较来构建此掩码，并且您应该利用 § 3.5.4 中缩放点积注意力实现已经支持注意力掩码这一事实。

应用绳索。RoPE 应该应用于查询向量和键向量，而不是值向量。此外，头部维度应该作为批次维度处理，因为在多头注意力机制中，注意力是独立应用于每个头部的。这意味着每个头部的查询向量和键向量都应该应用完全相同的 RoPE 旋转。

问题（多头自注意力）：实现因果多头自注意力机制（5分）

交付成果实现因果多头自注意力机制torch.nn.模块。您的实现应该（至少）接受以下参数：

d_model: 整数Transformer模块输入的维度。

头数: 整数多头自我关注训练中使用的头部数量。

根据 Vaswani 等人 [2017] 的研究，设置 $d_k = d_v = d_{\text{模型}} / h$ 要使用我们提供的测试用例来测试您的实现，请在以下位置实现测试适配器：`[adapters.run_multihead_self_attention]` 然后运行 `uv run pytest -k test_multihead_self_attention` 测试你的实现。

3.6 全变压器LM

我们首先来组装Transformer模块（参考图2会很有帮助）。一个Transformer模块包含两个“子层”，一个用于多头自注意力机制，另一个用于前馈网络。在每个子层中，我们首先执行RMSNorm操作，然后执行主要操作（MHA/FF），最后添加残差连接。

具体来说，Transformer 模块的前半部分（第一个“子层”）应该实现以下更新集以生成输出。是来自输入 x ，

$$y = x + \text{多头自注意力}(\text{RMSNorm}(x))。$$
 (15)

问题（transformer_block）：实现Transformer模块（3分）

按照 § 3.5 中的描述和图 2 中的图示实现预范数 Transformer 模块。您的 Transformer 模块应接受（至少）以下参数。

d_model: 整数Transformer模块输入的维度。

头数: 整数多头自我关注训练中使用的头部数量。

d_ff: 整数位置前馈内层的维度。

要测试你的实现，请实现适配器。`[adapters.run_transformer_block]`然后运行`uv run pytest -k test_transformer_block`测试你的实现。

交付成果：能够通过所提供测试的转换器代码块。

现在我们按照图 1 中的高级示意图将各个模块组合起来。请按照 3.1.1 节中对嵌入的描述，将其输入到层数 Transformer 模块，然后将其传递到三个输出层，以获得词汇表的分布。

问题 (transformer_lm)：实现Transformer LM (3分)

是时候将所有内容整合起来了！请按照 § 3.1 中的描述和图 1 中的图示实现 Transformer 语言模型。您的实现至少应接受 Transformer 模块的所有上述构造参数，以及以下附加参数：

词汇量： **整数** 词汇表的大小，对于确定词元的维度至关重要。
嵌入矩阵。

上下文长度： **整数** 确定维度所需的最大上下文长度
位置嵌入矩阵。

层数： **整数** 要使用的Transformer模块数量。

要使用我们提供的测试用例来测试您的实现，您首先需要实现测试适配器。`[adapters.run_transformer_lm]`然后运行`uv run pytest -k test_transformer_lm`测试你的实现。

交付成果：一个通过上述测试的 Transformer LM 模块。

资源核算。了解Transformer的各个部分如何消耗计算资源和内存非常有用。我们将逐步讲解一些基本的“浮点运算次数统计”方法。广阔的Transformer中大部分浮点运算都是矩阵乘法，因此我们的核心方法很简单：

1. 写出 Transformer 前向传播过程中所有的矩阵乘法运算。
2. 将每个矩阵乘法运算转换为所需的浮点运算次数。

对于第二步，以下事实将很有用：

规则： 鉴于一个 $A \in \mathbb{R}^{m \times n}$ 和 $B \in \mathbb{R}^{n \times p}$ 矩阵-矩阵乘积 AB 需要 $2np$ 浮点运算次数。

要理解这一点，请注意 $(AB)[i, j] = \sum_k A[i, k] \cdot B[k, j]$ ，并且这个点积要求 n 次乘法和 n 次加法（ $2n$ FLOPs）。然后，由于矩阵-矩阵乘积 AB 有 $m \times p$ 条目总数为 $(FLOPs) \ 2n(m) = 2mp$ 。

现在，在解决下一个问题之前，仔细检查一下 Transformer 模块和 Transformer LM 的每个组件，并列出了所有矩阵乘法及其相关的 FLOPs 成本，可能会有所帮助。

问题 (transformer_accounting): 变压器LM资源核算 (5分)

(a) 考虑 GPT-2 XL，其配置如下：

词汇量： 50,257

上下文长度： 1,024

层数： 48

d_model： 1600

头数: 25

d_{ff} : 6,400

假设我们使用此配置构建模型。我们的模型将有多少个可训练参数? 假设每个参数都用单精度浮点数表示, 那么仅加载此模型就需要多少内存?

交付成果用一到两句话回答。

- (b) 确定完成 GPT-2 XL 型模型前向传播所需的矩阵乘法运算。这些矩阵乘法运算总共需要多少 FLOPs? 假设我们的输入序列有上下文长度代币。

交付成果: 矩阵乘法列表 (附说明) 以及所需的总浮点运算次数。

- (c) 根据你上面的分析, 模型的哪些部分需要最多的 FLOPs? **交付成果**用一到两句话回答。

- (d) 使用 GPT-2 small (12 层, 768) 重复分析 d_{model} , 12 个头), GPT-2 中等 (24 层, 1024) d_{model} , (16 个头), 以及 GPT-2 大型 (36 层, 1280 个头) d_{model} , (20 个头部)。随着模型尺寸的增加, Transformer LM 的哪些部分占用的总 FLOPs 比例增加或减少?

交付成果对于每个模型, 请提供模型组件及其相关浮点运算次数 (占前向传播所需总浮点运算次数的比例) 的详细分解。此外, 请用一到两句话描述模型规模变化如何影响每个组件的浮点运算次数比例。

- (e) 将 GPT-2 XL 的上下文长度增加到 16,384。一次前向传播的总 FLOPs 值如何变化? 模型各组成部分的 FLOPs 相对贡献值如何变化?

交付成果用一到两句话回答。

4 训练 Transformer LM

现在我们已经完成了数据预处理（通过分词器）和模型预处理（Transformer）的步骤。接下来需要编写所有支持训练的代码。这包括以下内容：

- **损失**：我们需要定义损失函数（交叉熵）。
- **优化器**：我们需要定义优化器来最小化这种损失（AdamW）。
- **训练循环**：我们需要所有支持基础设施，用于加载数据、保存检查点和管理训练。

4.1 交叉熵损失

回想一下，Transformer 语言模型定义了一个分布 $p_\theta(x_{i+1}/x_{1:i})$ 对于每个序列 x 长度 $m+1$ 和 $i=1, \dots, m$ 给定一个训练集 D 由长度为 m 的序列组成我们定义标准交叉熵（负对数似然）损失函数：

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^m -\log p_\theta(x_{i+1}/x_{1:i}) \quad (16)$$

（注意，Transformer 中的一次前向传递会产生 $p_\theta(x_{i+1}/x_{1:i})$ 为了全部 $i=1, \dots, m$ 。）

具体来说，Transformer 计算 logits。 $o_i \in \text{拉词汇量}$ 每个位置 i 其结果为：⁶

$$p(x_{i+1}/x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{\text{一个}=1}^{\text{词汇量}} \exp(o_i[\text{一个}])} \quad (17)$$

交叉熵损失通常是相对于 logits 向量定义的。 $o_i \in \text{拉词汇量目标}$

⁷ x_{i+1} 。

实现交叉熵损失函数需要注意一些数值问题，就像 softmax 函数的情况一样。

问题（交叉熵）：实现交叉熵

交付成果 编写一个函数来计算交叉熵损失，该函数接受预测的 logits 作为输入 (o_i) 和目标 (x_{i+1}) 并计算交叉熵 $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$ 你的函数应该处理以下内容：

- 为了数值稳定性，减去最大元素。
- 尽可能抵消对数和经验值。
- 处理任何其他批次尺寸并返回平均的在整个批次中。与第 3.3 节一样，我们假设批次大小维度始终优先于词汇量大小维度。

实施 `[adapters.run_cross_entropy]` 然后运行 `uv run pytest -k test_cross_entropy` 测试你的实现。

困惑 交叉熵足以用于训练，但当我们评估模型时，我们也希望报告……

困惑。对于长度为 m 我们遭受交叉熵损失的地方 $1, \dots, m$ ：

$$\text{困惑度} = \exp \left(\frac{1}{m} \sum_{i=1}^m \ell_i \right) \quad (18)$$

⁶注意 $o_i[k]$ 指的是索引处的值 k 向量 o_i 。

⁷这对应于狄拉克 δ 分布之间的交叉熵 x_{i+1} 以及预测的 $\text{softmax}(o_i)$ 分配。

4.2 SGD 优化器

现在我们有损失函数，接下来我们将开始探索优化器。最简单的基于梯度的优化器是随机梯度下降（SGD）。我们从随机初始化的参数开始。 θ_0 然后对于每个步骤 $t=0, \dots, T-1$ ，我们执行以下更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

在哪里 B_t 是从数据集中随机抽取的一批数据 D 以及学习率 α_t 和批量大小 $|B_t|$ 是超参数。

4.2.1 在 PyTorch 中实现 SGD

为了实现我们的优化器，我们将继承 PyTorch 类。torch.optim.优化器班级。优化器 子类必须实现两个方法：

定义__init__（自己参数，...）应该初始化你的优化器。在这里，参数将是一系列待优化的参数（或参数组，如果用户希望对模型的不同部分使用不同的超参数，例如学习率）。请确保传递参数到__init__基类的方法，它将存储这些参数以供使用：步。您可以根据优化器（例如，学习率是一个常见的参数）接受其他参数，并将它们作为字典传递给基类构造函数，其中键是您为这些参数选择的名称（字符串）。

定义步（自己）应该对参数进行一次更新。在训练循环期间，这将被调用。

反向传播之后，您就可以访问最后一个批次的梯度。此方法应该遍历每个参数张量。p并对其进行修改就位即设置p.data，它保存着基于梯度的与该参数关联的张量p.grad（如果存在），表示损失函数相对于该参数的梯度的张量。

PyTorch 优化器 API 有一些微妙之处，因此用示例来解释会更容易。为了使我们的示例更丰富，我们将实现一个略有不同的 SGD 算法，其中学习率会在训练过程中衰减，初始学习率为 α 并随着时间的推移逐步采取更小的步骤：

$$\theta_{t+1} = \theta_t - \sqrt{\frac{\alpha}{t+1}} \nabla L(\theta_t; B_t) \quad (20)$$

让我们看看这个版本的 SGD 将如何用 PyTorch 实现。优化器：

从collections.abc进口可调用，可迭代 从打字进口选修的 进口火炬

进口数学

班级新加坡元（火炬。优化。优化器）：

定义__初始化__（自己参数，学习率=1e-3）：

如果左手<0：

增加值错误（f“无效的学习率：{左手}”）默认值={"lr": lr}

极好的()。__init__（参数，默认值）

定义步（自己，closure: Optional[Callable]=没有任何）：

损失=如果无关闭是没有其他关闭（）为了团体在自己。

参数组：

左手=团体["lr"]#获取学习率。

为了p在团体[参数]:

如果p。毕业生是没有任何:

继续

状态=自己。状态[p] t=状态 # 获取与 p 关联的状态。

态。得到 ("t" , 0) # 从状态或初始值中获取迭代次数。

毕业生=p。毕业生。数据 #求损失函数关于 p 的梯度。

p。数据=-左手/数学。sqrt(t+1) *毕业生 #原地更新权重张量。状态["t"]=t+1 #增加迭代次数。

返回损失

在 __init__，我们将参数以及默认超参数传递给基类构造函数，供优化器使用（参数可能以组的形式出现，每组参数都有不同的超参数）。如果参数只是一个集合，torch.nn.参数对于对象，基构造函数将创建一个组并为其分配默认超参数。然后，在步，我们遍历每个参数组，然后遍历该组中的每个参数，并应用公式 20。这里，我们将迭代次数作为与每个参数关联的状态：我们首先读取该值，将其用于梯度更新，然后更新该值。API 规定用户可以传入一个可调用对象。关闭在优化步骤之前重新计算损失。我们使用的优化器不需要这一步，但为了符合 API 规范，我们还是添加了这一步。

为了了解其工作原理，我们可以使用以下最小示例：训练循环:

重量=火炬。nn。范围 (5 *火炬。randn((10, 10))) 选项=SGD([权重], lr=1)

为了t在范围 (100) :

选择。零梯度() #重置所有可学习参数的梯度。损失= (重量) **2) 。意思是 () #计算标量损失值。打印 (损失。中央处理器 () 。物品 ())

损失。落后 () #运行反向传播，计算梯度。选择。步 () #运行优化器步骤。

这是训练循环的典型结构：在每次迭代中，我们将计算损失并运行优化器的一个步骤。在训练语言模型时，我们的可学习参数将来自模型（在 PyTorch 中，m.parameters()（给我们提供这个集合）。损失将基于采样批次的数据进行计算，但训练循环的基本结构将保持不变。

问题（学习率调优）：调整学习率（1 分）

正如我们将看到的，影响训练效果最大的超参数之一是学习率。让我们通过一个简单的例子来实际观察一下。运行上面的 SGD 示例，分别使用三个不同的学习率值：1e1、1e2 和 1e3，训练迭代次数仅为 10 次。对于每个学习率，损失函数会发生什么变化？它是衰减得更快、更慢，还是发散（即在训练过程中不断增加）？

交付成果请用一两句话描述您观察到的行为。

4.3 AdamW

现代语言模型通常使用更复杂的优化器进行训练，而不是随机梯度下降（SGD）。最近使用的大多数优化器都是 Adam 优化器的衍生算法 [Kingma and Ba, 2015]。我们将使用 AdamW [Loshchilov and Hutter, 2019]，它在近期的研究中得到了广泛应用。AdamW 对 Adam 进行了改进，通过添加参数来增强正则化效果。体重下降（在每次迭代中，我们将参数拉向0），

以一种与梯度更新解耦的方式。我们将按照 Loshchilov 和 Hutter [2019] 的算法 2 中描述的方式实现 AdamW。

AdamW 是有状态的对于每个参数，它都会跟踪其一阶矩和二阶矩的运行估计值。因此，AdamW 使用额外的内存来换取更高的稳定性和收敛性。此外，学习率也很重要。 α AdamW 有一对超参数 (β_1, β_2) 控制矩估计更新的参数以及权重衰减率 λ 。典型应用集 (β_1, β_2) 到 $(0.9, 0.999)$ ，但像 LLaMA [Touvron 等人, 2023] 和 GPT-3 [Brown 等人, 2020] 这样的大型语言模型通常是用以下方式训练的： $(0.9, 0.95)$ 。该算法可以写成如下形式，其中 ϵ 是一个较小的值（例如， 10^{-8} 用于在数值极小时提高数值稳定性 ν ）：

算法 1 AdamW 优化器

初始化 θ (初始化可学习参数)

$m \leftarrow 0$ (一阶矩向量的初始值；形状与 θ) $v \leftarrow 0$ (二阶矩向量的初始值；形状与 θ) 为了 $t=1, \dots, T$ 做

数据样本批次 B_t

$g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$ 计算当前时间步损失函数的梯度 $m \leftarrow \beta_1 m + (1 - \beta_1) g$ (更新一阶矩估计值) $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$ (更新二阶矩估计值)

$\alpha_t \leftarrow \alpha \frac{(1 - \beta_2^t)}{(1 - \beta_1^t)}$ (计算调整后) α 迭代 t (更新参数)

$\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v} + \epsilon}$

$\theta \leftarrow \theta - \alpha \lambda \theta$ (应用重量衰减) 结束

注意 t 开始于 1。接下来，您将实现这个优化器。

问题 (adamw)：实现 AdamW (2 分)

交付成果：将 AdamW 优化器实现为以下类的子类 `torch.optim.优化器`。你们班应该学习率 α 在 `__init__`，以及 β ， ϵ 和 λ 超参数。为了帮助您维护状态，基本优化器课堂会给你一本字典自我状态，哪些地图 `nn.Parameter` 将对象转换为字典，该字典存储该参数所需的任何信息（对于 AdamW，这将是矩估计值）。实现 `adapters.get_adamw_cls` 并确保它通过。uv 运行 `pytest -k test_adamw`。

问题 (adamwAccounting)：使用 AdamW 进行培训的资源核算 (2 分)

让我们计算一下运行 AdamW 需要多少内存和计算资源。假设我们对每个张量都使用 `float32` 类型。

- (a) 运行 AdamW 需要多少峰值内存？请根据参数、激活值、梯度和优化器状态的内存使用情况分解你的答案。用以下方式表示你的答案：批次大小以及模型超参数 (`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`)。认为 `d_ff = 4 * d_model`。

为简化起见，在计算激活的内存使用量时，仅考虑以下组成部分：

- 变压器模块
 - 均方根值

- 多头自注意力子层: QKV 预测, QK 矩阵乘法、softmax、加权值求和、输出投影。

- 位置前馈: W_1 矩阵乘法, SiLU, W_2 矩阵乘法

- 最终 RMSNorm
- 输出嵌入
- 对数坐标的交叉熵

交付成果: 每个参数、激活值、梯度和优化器状态以及总和的代数表达式。

(b) 将你的答案实例化为 GPT-2 XL 形状模型, 以获得仅依赖于以下因素的表达式: 批次大小。在80GB内存范围内, 最大批处理大小是多少?

交付成果 一个看起来像这样的表达式 $a \cdot \text{批次大小} + b$ 对于数值 a , b 以及表示最大批次大小的数字。

(c) AdamW 运行一步需要多少 FLOPs? **交付成果:** 一个代数表达式, 并附简要说明。

(d) 模型浮点运算利用率 (MFU) 定义为观测吞吐量 (每秒令牌数) 与硬件理论峰值浮点运算吞吐量的比值 [Chowdhery et al., 2022]。NVIDIA A100 GPU 的理论峰值浮点运算吞吐量为 19.5 teraFLOP/s (针对 float32 运算)。假设 MFU 为 50%, 那么在单个 A100 上训练 GPT-2 XL 模型, 训练 40 万步, 批大小为 1024, 需要多长时间? 参考 Kaplan et al. [2020] 和 Hoffmann et al. [2022] 的研究, 假设反向传播的浮点运算量是正向传播的两倍。

交付成果: 培训所需天数, 并简要说明理由。

4.4 学习率调度

训练过程中, 能够使损失函数快速下降的学习率值通常会发生变化。在训练Transformer模型时, 通常会使用一个学习率。日程我们一开始会使用较大的学习率, 在初期进行更快速的更新, 然后随着模型的训练, 逐渐将其衰减到一个较小的值。⁸

在本任务中, 我们将实现用于训练 LLaMA 的余弦退火策略 [Touvron 等人, 2023]。

调度器就是一个接收当前步骤的函数。它以及其他相关参数 (例如初始学习率和最终学习率), 并返回在步骤中用于梯度更新的学习率。最简单的学习率调整方法是使用常数函数, 它对于任何给定的学习率都返回相同的学习率。 t_0

余弦退火学习率调度考虑了 (i) 当前迭代 t (ii) 最大学习率 $\alpha_{\text{最大限度}}$ (三) 最小 (最终) 学习率 $\alpha_{\text{最小}}$ (iv) 数量热身迭代 $T_{\text{西}}$, 以及 (v) 余弦退火迭代次数 T_c 迭代中的学习率 α 定义为:

(热身) 如果 $t < T_{\text{西}}$, 然后 $\alpha = t / T_{\text{西}} \alpha_{\text{最大限度}}$ 。

(余弦退火) 如果 $T_{\text{西}} \leq t \leq T_c$, 然后 $\alpha = \alpha_{\text{最小}} + \frac{1}{2} (\alpha_{\text{最大限度}} - \alpha_{\text{最小}}) \left(1 + \cos \left(\frac{t - T_{\text{西}}}{T_c - T_{\text{西}}} \pi \right) \right)$ 。

(退火后) 如果 $t > T_c$, 然后 $\alpha = \alpha_{\text{最小}}$ 。

⁸有时, 为了帮助学生突破局部最小值, 会采用一种让学习率逐步提高 (重新启动) 的计划。

问题（学习率计划）：实现余弦学习率调度并进行预热

编写一个接受以下参数的函数 t , $\alpha_{\text{最大限度}}$, $\alpha_{\text{最小}}$, $T_{\text{预热}}$ 和 $T_{\text{总}}$ 并返回学习率 α_t 根据上面定义的调度器, 然后实现。[adapters.get_lr_cosine_schedule]并确保它通过。uv 运行 `pytest -k test_get_lr_cosine_schedule`。

4.5 渐变裁剪

在训练过程中, 我们有时会遇到梯度较大的训练样本, 这会导致训练不稳定。为了缓解这种情况, 实践中常用的一种技术是: **渐变裁剪** 其思路是在每次反向传播之后、执行优化器步骤之前, 对梯度范数施加限制。

给定梯度 (对于所有参数) g , 我们计算它的 ℓ_2 -范数 (或 $\|g\|_2$)。如果该标准小于最大值 M 然后我们就离开 g 按原样进行, 否则, 我们进行缩放。克下降了 $\frac{M}{\|g\|_2}$ (一个 ϵ , 我们喜欢 10^{-6} (添加是为了数值稳定性))。注意, 最终得到的范数将略小于 M 。

问题（gradient_clipping):实现渐变裁剪（1分）

编写一个实现梯度裁剪的函数。该函数应接受一个参数列表和一个最大值。 ℓ_2 -norm。它应该就地修改每个参数的梯度。使用 $\epsilon=10^{-6}$ (PyTorch 默认值)。然后, 实现适配器 [adapters.run_gradient_clipping]并确保它通过。uv 运行 `pytest -k test_gradient_clipping`。

5 训练循环

现在，我们将最终把目前为止构建的主要组件组合在一起：标记化数据、模型和优化器。

5.1 数据加载器

标记化数据（例如，您在……中准备的数据）tokenizer_experiments)是单个标记序列 $x = (x_1, \dots, x_n)$ 即使源数据可能包含多个单独的文档（例如，不同的网页或源代码文件），常见的做法是将所有这些文档连接成一个单一的标记序列，并在它们之间添加分隔符（例如 `<|>`）。文本结束`>`令牌）。

一个数据加载器将其转化为一条流批次其中每批包含 B 长度为的序列 m 与相应的下一个标记配对，长度也相同。 m 例如，对于 $B=1$ ， $m=3$ ， $[(x_2, x_3, x_4), (x_3, x_4, x_5)]$ 这可能是其中一批。

以这种方式加载数据可以简化训练过程，原因有很多。首先，任何 $1 \leq i < n - m$ 给出了一个有效的训练序列，因此序列采样非常简单。由于所有训练序列的长度都相同，所以无需填充输入序列，这提高了硬件利用率（也通过增加批处理大小来实现）。最后，我们也不需要完全加载整个数据集来对训练数据进行采样，这使得处理可能无法放入内存的大型数据集变得容易。

问题（数据加载）：实现数据加载（2分）

交付成果： 编写一个接受 NumPy 数组作为参数的函数 x （包含标记 ID 的整数数组），批次大小，一个上下文长度以及一个 PyTorch 设备字符串（例如，“中央处理器”或者 `'cuda:0'`），并返回一对张量：采样后的输入序列和对应的下一个标记目标。两个张量的形状都应该是（批次大小，context_length）包含令牌 ID，两者都应放置在请求的设备上。要使用我们提供的测试用例测试您的实现，您首先需要实现测试适配器。`[adapters.run_get_batch]` 然后运行 `uv run pytest -k test_get_batch` 测试你的实现。

低资源/缩减配置技巧：在 CPU 或 Apple Silicon 上加载数据

如果您计划在 CPU 或 Apple Silicon 上训练您的 LM，则需要将数据移动到正确的设备（同样，您以后也应该使用同一设备来训练您的模型）。

如果你使用的是 CPU，你可以使用“中央处理器”设备字符串，在 Apple Silicon（M* 芯片）上，您可以使用“议员们设备字符串。

有关 MPS 的更多信息，请查看以下资源：

- <https://developer.apple.com/metal/pytorch/>
- <https://pytorch.org/docs/main/notes/mps.html>

如果数据集太大，无法加载到内存中怎么办？我们可以使用一个名为 `'fs'` 的 Unix 系统调用。mmap 它将磁盘上的文件映射到虚拟内存，并在访问该内存位置时延迟加载文件内容。因此，您可以“假装”已将整个数据集加载到内存中。Numpy 通过以下方式实现此功能：`np.memmap`（或者旗帜 `mmap_mode='r'` 到 `np.load`，如果您最初保存数组时使用了 `np.save`），它将返回一个类似 numpy 数组的对象，该对象会在您访问时按需加载条目。在训练过程中从数据集（例如，numpy 数组）中采样时，请确保以内存映射模式加载数据集。（通过 `np.memmap` 或者旗帜 `mmap_mode='r'` 到 `np.load`，取决于你保存数组的方式）。确保你也指定一个数据类型这与您正在加载的数组相匹配。显式验证内存映射的数据是否正确（例如，不包含超出预期词汇表大小的值）可能会有所帮助。

5.2 检查点

除了加载数据之外，我们还需要保存训练过程中的模型。运行作业时，我们通常希望能够恢复因某种原因（例如作业超时、机器故障等）中途停止的训练。即使一切顺利，我们之后也可能需要访问中间模型（例如，事后研究训练动态、从不同训练阶段的模型中抽取样本等）。

检查点应该包含恢复训练所需的所有状态。我们当然至少需要能够恢复模型权重。如果使用有状态优化器（例如 AdamW），我们还需要保存优化器的状态（例如，对于 AdamW，需要保存矩估计值）。最后，为了恢复学习率调度，我们需要知道停止训练时的迭代次数。PyTorch 可以轻松保存所有这些信息：每个 nn 模块有一个 `state_dict()` 该方法返回一个包含所有可学习权重的字典；我们稍后可以使用姊妹方法恢复这些权重。加载状态字典()。任何事物都是如此。nn.optim.优化器。最后，`torch.save(obj, dest)` 可以将对象（例如，包含某些值的张量字典，以及像整数这样的常规 Python 对象）转储到文件（路径）或类文件对象，然后可以使用以下命令将其加载回内存：`torch.load(src)`。

问题（检查点）：实现模型检查点机制（1 分）

实现以下两个函数来加载和保存检查点：

定义保存检查点（模型、优化器、迭代、输出） 应该将所有状态从系统中清除。

将前三个参数输入到类文件对象中出去。你可以使用状态字典模型和优化器获取其相关状态并使用的方法火炬。保存(obj, out)倾倒入对象输入/输出（PyTorch 支持路径或类似文件的对象）。一种常见的选择是：对象可以是字典格式，但只要以后可以加载检查点，你可以使用任何你想要的格式。

此函数需要以下参数：

型号：手电筒。nn。模块优化器：

torch。优化。优化器迭代：整数

出去：斯特|操作系统。路径类|打字。二进制输入/输出|打字。IO[字节]

定义加载检查点（源、模型、优化器） 应该从此处加载检查点src（路径或文件

（例如对象），然后从该检查点恢复模型和优化器状态。您的函数应返回保存到检查点的迭代次数。您可以使用 `torch.load(src)` 恢复您在账户中保存的内容保存检查点实施，以及 加载状态字典模型和优化器中都有使它们恢复到先前状态的方法。

此函数需要以下参数：

来源：斯特|操作系统。路径类|打字。二进制输入/输出|打字。IO[字节型]

号：火炬。nn。模块优化器：torch。优化。优化器

实施[`adapters.run_save_checkpoint`]和[`adapters.run_load_checkpoint`]适配器，并确保它们通过uv 运行 `pytest -k test_checkpointing`。

5.3 训练循环

现在，终于到了将你实现的所有组件整合到主训练脚本中的时候了。这样做的好处在于，你可以很方便地使用不同的超参数启动训练（例如，通过命令行参数传递参数），因为之后你会多次执行这些操作来研究不同的选择如何影响训练结果。

问题（一起训练）：把它们组合起来（4分）

交付成果编写一个脚本，运行训练循环，使用用户提供的输入来训练模型。我们特别建议训练脚本至少应包含以下内容：

- 能够配置和控制各种模型和优化器的超参数。
- 以内存高效的方式加载训练和验证的大型数据集`np.memmap`。
- 将检查点序列化到用户提供的路径。
- 定期记录训练和验证性能（例如，记录到控制台和/或外部服务，如 Weights and Biases）。一个

— wandb.ai

6 生成文本

既然我们已经可以训练模型，那么我们需要做的最后一件事就是让模型生成文本。回想一下，语言模型接收一个长度为 (t) 的（可能是批量处理的）整数序列。序列长度 (t) 并生成一个大小为 $(\text{序列长度} \times \text{词汇量})$ ，其中序列中的每个元素都是一个概率分布，用于预测该位置之后的下一个单词。接下来，我们将编写一些函数，将其转换为新序列的采样方案。

Softmax按照惯例，语言模型的输出是最后一个线性层的输出（“logits”），因此我们需要通过以下方式将其转换为归一化概率：**软麦克斯**操作，我们在前面的公式 10 中已经看到了。

解码为了从我们的模型生成文本（解码），我们将向模型提供一个前缀标记序列（“提示”），并要求它生成一个词汇表概率分布，用于预测序列中的下一个词。然后，我们将从该词汇表分布中进行采样，以确定下一个输出标记。

具体来说，解码过程的一个步骤应该包含以下几个步骤： $x_1, \dots, x_t$ 并返回一个令牌 x_{t+1} 通过以下公式，

$$P(x_{t+1}=i/x_1, \dots, x_t) = \frac{\exp(v_i)}{\sum_{j \in \text{词汇量}} \exp(v_j)}$$

$v = \text{TransformerLM}(x_1, \dots, x_t) \quad t \in \text{拉词汇量}$

其中 TransformerLM 是我们的模型，它以序列作为输入：序列长度并生成一个大小为 $(\text{序列长度} \times \text{vocab_size})$ 我们取该矩阵的最后一个元素，因为我们正在寻找下一个单词的预测结果。 t 第-位置。

这为我们提供了一个基本的解码器，方法是反复从这些单步条件中采样（将我们先前生成的输出标记附加到下一个解码时间步的输入），直到生成序列结束标记 `<|endoftext>` (或用户指定的最大生成令牌数)。

解码器技巧我们将尝试使用小型模型，而小型模型有时会生成质量非常低的文本。两个简单的解码器技巧可以帮助解决这些问题。首先，在**温度标度**我们使用温度参数修改 softmax 函数 τ 其中新的softmax是

$$\text{softmax}(v, \tau) = \frac{\exp(v_i / \tau)}{\sum_{j=1}^{\text{词汇量}} \exp(v_j / \tau)} \quad (24)$$

注意设置方式 $\tau \rightarrow 0$ 使得最大元素 v 占主导地位，softmax 的输出变成一个集中在该最大元素上的独热向量。

其次，还有另一个技巧是**核**或者**顶级球员**抽样，其中我们通过截断低概率词来修改抽样分布。 q 是我们从大小为 (温度缩放的) softmax 得到的概率分布 (vocab_size) 。超参数核采样 p 根据生成下一个令牌

方程

$$P(x_{t+1}=i/q) = \begin{cases} \frac{\sum_{j \in V(p)} q_j}{\sum_{j \in V(p)} q_j} & \text{如果 } i \in V(p) \\ 0 & \text{否则} \end{cases}$$

在哪里 $V(p)$ 是**最小**一组索引，使得 $\sum_{j \in V(p)} q_j \geq p$ 你可以通过以下方式轻松计算出这个量：首先对概率分布进行排序 q 按数量级，选择最大的词汇元素，直到达到目标水平。 α 。

问题（解码）：解码（3分）

交付成果实现一个用于从语言模型解码的函数。我们建议您支持以下功能：

- 根据用户提供的提示生成补全答案（例如，接收一些 $x_1, \dots, x_t$ 并采样完成情况，直到遇到 <|文本结束|>令牌）。
- 允许用户控制生成的令牌的最大数量。
- 给定一个期望的温度值，在采样之前对预测的下一个词分布应用 softmax 温度缩放。
- Top-p 抽样（Holtzman 等人，2020；也称为核抽样），给定用户指定的阈值。

7个实验

现在是时候将所有内容整合起来，并在相关的数据集上训练（小型）语言模型了。

7.1 如何开展实验和交付成果

要理解Transformer架构组件背后的原理，最好的方法就是实际修改并运行它。实践经验无可替代。

为此，具备实验能力至关重要。**快速、持续地并保留记录**为了快速进行实验，我们将使用小型模型（1700 万个参数）和简单数据集（TinyStories）进行大量实验。为了保证实验的一致性，您需要系统地消融模型组件并调整超参数。为了记录实验过程，我们将要求您提交实验日志以及与每个实验相关的学习曲线。

为了能够提交损失曲线，**务必定期评估验证损失，并记录步骤数和实际耗时**。您可能会发现诸如 Weights and Biases 之类的日志记录基础设施很有用。

问题（实验日志）：实验记录（3分）

对于您的训练和评估代码，创建实验跟踪基础架构，以便您可以跟踪实验和损失曲线与梯度步长和实际运行时间的关系。

交付成果：记录实验的基础设施代码，以及本节中以下作业问题的实验日志（记录你尝试过的所有事情的文档）。

7.2 微型故事

我们将从一个非常简单的数据集（TinyStories；Eldan 和 Li，2023）开始，模型可以快速训练，并且我们可以观察到一些有趣的现象。获取此数据集的说明请参见第 1 节。数据集的示例如下所示。

例子（tinystories_example):TinyStories 的一个例子

从前，有个小男孩名叫本。本喜欢探索周围的世界。他看到了很多奇妙的东西，比如商店里陈列的漂亮花瓶。一天，本在商店里闲逛时，偶然发现了一个非常特别的花瓶。他看到后惊叹不已！他说：“哇，这个花瓶真漂亮！我可以买吗？”店主笑着说：“当然可以。你可以把它带回家，让你的朋友们看看它有多漂亮！”于是，本把花瓶带回了家，他为此感到非常自豪！他叫来朋友们，向他们展示这个漂亮的花瓶。他的朋友们都觉得花瓶很漂亮，并且难以置信本竟然这么幸运。这就是本在商店里发现漂亮花瓶的故事！

超参数调优我们会先告诉你一些非常基本的超参数，然后请你找到一些其他效果良好的设置。

词汇量10000。典型的词汇量在数万到数十万之间。你应该根据实际情况调整这个数字。

观察词汇和模型行为的变化。

上下文长度256. 像 TinyStories 这样的简单数据集可能不需要很长的序列长度，但是对于

对于后续的 OpenWebText 数据，您可能需要调整此值。尝试调整此值，并观察其对每次迭代运行时间和最终困惑度的影响。

d_model512. 这比许多小型Transformer论文中使用的768尺寸略小，但是这样可以加快速度。

d_ff1344年。这大约是8 $\times$ **d_model**同时又是 64 的倍数，这对 GPU 性能有好处。

RoPE theta 参数 Θ 10000。

层数和头部数量4层，16个喷头。加起来，这将产生大约1700万个非包埋性颗粒。参数，这是一个相当小的变压器。

已处理令牌总数327,680,000（您的批次大小） $\times$ 总步数 $\times$ 上下文长度应等于大约是这个值）。

您应该通过反复试验来找到以下其他超参数的合适默认值：学习率、学习率预热、其他 AdamW 超参数（ β_1 ， β_2 ， ϵ ），和体重下降。您可以在 Kingma 和 Ba [2015] 中找到一些此类超参数的典型选择。

把它们组合起来现在，你可以通过获取训练好的 BPE 分词器、对训练数据集进行分词，并在你编写的训练循环中运行它，将所有内容整合在一起。**重要提示：**如果你的实现正确且高效，上述超参数在单块 H100 GPU 上应该能达到大约 30-40 分钟的运行时间。如果运行时间过长，请检查并确保你的数据加载、检查点或验证损失代码没有成为运行瓶颈，并确保你的实现正确使用了批处理。

模型架构调试技巧和窍门我们强烈建议您熟悉 IDE 的内置调试器（例如 VSCode/PyCharm），与使用 print 语句调试相比，这将节省您的时间。如果您使用文本编辑器，则可以使用类似这样的工具：pdb。调试模型架构时，还有一些其他的良好实践：

- 开发任何神经网络架构时，常见的第一步是对单个小批量数据进行过拟合。如果你的实现正确，你应该能够迅速将训练损失降至接近于零。
- 在各种模型组件中设置调试断点，并检查中间张量的形状，以确保它们符合您的预期。
- 监控激活值、模型权重和梯度的规范，确保它们不会爆炸或消失。

问题（学习率）：调整学习率（3 分）（4 小时/100 小时）

学习率是需要调整的最重要的超参数之一。基于你训练好的基础模型，回答以下问题：

(a) 对学习率进行超参数扫描，并报告最终损失（如果优化器发散，则记录发散情况）。

交付成果：与多个学习率相关的学习曲线。解释你的超参数搜索策略。

交付成果在 TinyStories 数据集上，验证损失（每个 token）至多为 1.45 的模型

低资源/降级训练技巧：先在 CPU 或 Apple Silicon 上进行几步训练。

如果您正在运行中央处理器或者议员们，你应该将处理的总令牌数减少到40,000,000，这样就足以生成流畅度尚可的文本。您也可以将目标验证损失从 1.45 增加到 2.00。

我们在配备 36 GB 内存的 M3 Max 芯片上运行我们的解决方案代码，并使用调整后的学习率，批处理大小为 $\times$ 总步数 $\times$ 上下文长度 $= 32 \times 5000 \times 256 = 40,960,000$ 代币，耗时1小时22分钟中央处理器以及36分钟议员们。在第 5000 步，我们实现了 1.80 的验证损失。

一些补充建议：

- 使用时 λ 我们建议调整余弦学习率衰减策略，使其在第步终止衰减（即达到最小学习率）。 λ_0
- 使用时议员们，做**不是**使用 TF32 内核，即执行**不是放**火炬。设置 `float32_matmul_precision('高的')`
就像你可能会……库达我们尝试在设备上启用 TF32 内核。mps（火炬）版本 2.6.0 并发现后端将使用静默损坏的内核，导致训练不稳定。
- 您可以通过对模型进行即时编译来加快训练速度。`torch.compile`。具体来说：
 - 在中央处理器，使用以下命令编译您的模型
模型=火炬。编译（模型）
 - 在议员们，您可以使用以下方法对反向传播进行一些优化：
模型=火炬。编译（模型，后端）=`"aot_eager"`
不支持使用电感器进行编译议员截至火炬版本 2.6.0。

(b) 民间智慧认为，最佳学习速度处于“稳定边缘”。探究学习速度开始发散的点与你的最佳学习速度之间的关系。

交付成果：学习率递增的学习曲线，其中至少包含一次发散运行，并分析这与收敛率的关系。

现在我们来改变批次大小，看看训练结果会如何。批次大小很重要——它能让我们通过执行更大的矩阵乘法来提高GPU的效率，但我们真的总是需要很大的批次大小吗？让我们做一些实验来找出答案。

问题（batch_size_experiment):批次大小变化（1分）（2小时 100小时）

将批处理大小从 1 调整到 GPU 内存限制。尝试几个中间的批处理大小，包括 64 和 128 等常见大小。

交付成果：不同批次大小运行的学习曲线。如有必要，应再次优化学习率。

交付成果请用几句话讨论一下您关于批次大小及其对训练的影响的研究结果。

有了解码器，我们现在就可以生成文本了！我们将使用模型生成文本，看看效果如何。作为参考，您应该得到至少与以下示例一样好的输出结果。

例子 (ts_generate_example) : TinyStories 语言模型的示例输出

从前，有一个漂亮的小女孩名叫莉莉。她喜欢吃口香糖，尤其是那种又大又黑的。有一天，莉莉的妈妈让她帮忙做晚饭。莉莉兴奋极了！她喜欢帮妈妈的忙。莉莉的妈妈煮了一大锅汤。莉莉高兴极了，说：“谢谢妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。晚饭后，莉莉的妈妈又做了一些美味的汤。莉莉很喜欢！她说：“谢谢妈妈！这汤真好喝！”妈妈笑着说：“莉莉，你喜欢就好。”她们一起做完了饭，然后继续一起做饭。故事结束。

低资源/缩减空间技巧：在 CPU 或 Apple Silicon 上生成文本

如果使用低资源配置，处理 4000 万个词元，则生成的文本仍然类似于英语，但不如上述流畅。例如，我们使用 4000 万个词元训练的 TinyStories 语言模型的示例输出如下：

从前，有个小女孩名叫苏。苏有一颗她非常珍爱的牙齿，那是她最喜欢的牙齿。一天，苏出去散步，遇到了一只瓢虫！她们成了好朋友，一起在小路上玩耍。

“嘿，波莉！我们出去吧！”蒂姆说。苏抬头望向天空，发现很难找到一条闪耀的舞步。她笑了笑，答应帮忙说话！

苏看着天空移动，想知道那是什么。

以下是问题的具体描述以及我们的要求：

问题（产生）：生成文本（1 分）

使用解码器和训练好的检查点，报告模型生成的文本。您可能需要调整解码器参数（温度、top-p 等）以获得流畅的输出。

交付成果：至少包含 256 个文本标记的文本转储（或直到第一个 <|文本结束|>（标记），并简要评论此输出的流畅性以及至少两个影响此输出好坏的因素。

7.3 消融术和结构改造

理解Transformer的最佳方法是实际修改它并观察其行为。接下来，我们将进行一些简单的消融和修改。

消融 1：层归一化人们常说层归一化对于Transformer训练的稳定性至关重要。但或许我们想冒险一试。让我们从每个Transformer模块中移除RMSNorm，看看会发生什么。

问题 (layer_norm_ablation):移除 RMSNorm 并进行训练（1 分）（1 小时 100 小时）

从Transformer模型中移除所有RMSNorm，然后重新训练。在之前的最佳学习率下会发生什么？使用更低的学习率能否获得稳定性？

交付成果：移除 RMSNorms 并进行训练的学习曲线，以及最佳学习率的学习曲线。

交付成果对 RMSNorm 的影响做几句话的评论。

现在让我们来研究另一个乍看之下似乎任意的层归一化选择。*前规范*转换器模块定义为：

$$z = x + \text{多头自注意力}(\text{RMSNorm}(x)) \quad \text{是} \quad z + \text{FFN}(\text{RMSNorm}(z))。$$

这是对原始Transformer架构为数不多的“共识性”修改之一，该架构使用了一种*后规范*方法是

$$z = \text{RMSNorm}(x + \text{多头自注意力}(x)) \quad \text{是} \quad \text{RMSNorm}(z + \text{FFN}(z))。$$

让我们回到……*后规范*走过去看看会发生什么。

问题 (pre_norm_ablation): 实施后规范并进行培训 (1 分) (1 小时 100 小时)

将你的预归一化Transformer实现修改为后归一化版本。使用后归一化模型进行训练，看看会发生什么。

交付成果与预范数转换器相比，后范数转换器的学习曲线更陡峭。

我们发现，层归一化对转换器的行为有重大影响，甚至层归一化的位置也很重要。

消融术 2：位置嵌入接下来，我们将研究位置嵌入对模型性能的影响。具体来说，我们将比较我们的基础模型（包含 RoPE）和完全不包含位置嵌入的模型（NoPE）。结果表明，仅解码器 Transformer（即我们实现的带有因果掩码的模型）理论上可以在不显式提供位置嵌入的情况下推断相对或绝对位置信息 [Tsai et al., 2019, Kazemnejad et al., 2023]。现在，我们将通过实证测试 NoPE 与 RoPE 的性能对比。

问题 (no_pos_emb): 实施 NoPE (1 分) (1 小时 100 小时)

使用 RoPE 修改你的 Transformer 实现，完全移除位置嵌入信息，看看会发生什么。

交付成果：比较 RoPE 和 NoPE 性能的学习曲线。

消融术 3：SwiGLU 与 SiLU 的比较接下来，我们将效仿 Shazeer [2020]，通过比较 SwiGLU 前馈网络与使用 SiLU 激活但没有门控线性单元 (GLU) 的前馈网络的性能，来测试门控在前馈网络中的重要性：

$$\text{FFN}_{\text{思路}}(x) = W_2 \text{SiLU}(W_1 x)。 \quad (25)$$

回想一下，在我们的 SwiGLU 实现中，我们将内部前馈层的维度设置为大约 $d_{\text{ff}}=8 \times d_{\text{模型}}$ （同时确保 $d_{\text{ff}} \bmod 64 = 0$ ，利用 GPU 张量核心）。在您的 FFN_{思路}你应该设置的实现 $d_{\text{ff}}=4 \times d_{\text{模型}}$ ，以大致匹配 SwiGLU 前馈网络的参数数量（该网络有三个而不是两个权重矩阵）。

问题 (swiglu_ablation)：SwiGLU 对阵 SiLU (1 分) (1 小时 100 小时)

交付成果：比较 SwiGLU 和 SiLU 前馈网络性能的学习曲线，参数数量大致匹配。

交付成果 请用几句话概括您的发现。

资源不足/降级技巧：GPU 资源有限的在线学生应在 TinyStories 上测试修改。

在本作业的剩余部分，我们将转向更大规模、噪声更大的网络数据集（Open-WebText），尝试架构修改，并（可选地）提交到课程排行榜。

在 OpenWebText 上训练语言模型达到流畅水平需要很长时间，因此我们建议 GPU 访问有限的在线学生继续在 TinyStories 上测试修改（使用验证损失作为评估性能的指标）。

7.4 在 OpenWebText 上运行

接下来，我们将使用通过网络爬虫创建的更标准的预训练数据集。我们还提供了一个 OpenWebText [Gokaslan et al., 2019] 的小样本，它以单个文本文件的形式提供：有关如何访问此文件，请参见第 1 节。

以下是 OpenWebText 中的一个例子。请注意，这里的文本更加真实、复杂且多样化。您可以查看训练数据集，了解网络爬虫语料库的训练数据是什么样的。

例子（owt_example）：OWT 的一个例子

《棒球展望》（Baseball Prospectus）的技术总监哈里·帕夫利迪斯（Harry Pavlidis）聘请乔纳森·贾奇（Jonathan Judge）可谓冒了很大的风险。帕夫利迪斯深知，正如艾伦·施瓦茨（Alan Schwarz）在《数字游戏》（The Numbers Game）一书中写道：“在美国文化中，没有哪个领域比棒球运动员的表现更能被精确计算、更被热情地量化。”只需轻点几下鼠标，你就能发现诺亚·辛德加德（Noah Syndergaard）的快速球在飞向本垒板的过程中每分钟旋转超过2100圈，尼尔森·克鲁兹（Nelson Cruz）在2016年拥有所有合格击球手中最高的平均击球初速，以及其他无数看似来自电子游戏或科幻小说的奇闻轶事。海量数据的涌现赋予了棒球文化中一个日益重要的角色——数据分析爱好者——以力量。

这种权力赋予了他们更多关注——不仅关注数据本身，也关注数据背后的团队和出版物。帕夫利迪斯在创办《棒球展望》（Baseball Prospectus）时，深知量化数据不完善会招致怎样的强烈反对。他也明白网站的接球数据需要改进，而这需要一位博学之士——一位能够解决复杂统计建模问题的人。

完成这项工作。

“他把我们都吓坏了。”——哈里·帕夫利迪斯

帕夫利迪斯根据贾奇的写作以及两人在一次由球场主办的活动上的互动，预感贾奇“明白了”这个职位的用意。不久之后，两人边喝边聊。帕夫利迪斯的直觉得到了证实。贾奇非常适合这个职位——更重要的是，他欣然接受了这份工作。“我和很多人谈过，”帕夫利迪斯说，“只有他有勇气接受这份工作。” [...]

笔记：对于此实验，您可能需要重新调整超参数，例如学习率或批次大小。

问题（主实验）：OWT实验（2分）（3小时100小时）

使用与 TinyStories 相同的模型架构和总训练迭代次数，在 OpenWebText 上训练您的语言模型。该模型的表现如何？

交付成果：请描述您的语言模型在 OpenWebText 上的学习曲线。请说明它与 TinyStories 相比损失函数的差异——我们应该如何解读这些损失函数？

交付成果：使用 OpenWebText LM 生成的文本，格式与 TinyStories 的输出相同。这段文本的流畅度如何？为什么我们使用了与 TinyStories 相同的模型和计算资源，输出质量却更差？

7.5 您自己的修改 + 排行榜

恭喜你走到这一步！你快要完成了！现在，你将尝试改进Transformer架构，并看看你的超参数和架构与班上其他同学相比如何。

排行榜规则除以下限制外，没有其他限制：

运行时您的提交内容在 H100 上最多可以运行 1.5 小时。您可以通过设置来强制执行此限制。

-- 时间=01:30:00在你的 Slurm 提交脚本中。

数据您只能使用我们提供的 OpenWebText 训练数据集。

除此之外，你可以随心所欲地做任何你想做的事。

如果您正在寻找一些实施方案方面的想法，可以参考以下资源：

- 最先进的开源 LLM 系列，例如 Llama 3 [Grattafiori 等人，2024] 或 Qwen 2.5 [Yang 等人，2024]。
- NanoGPT 速通库 (<https://github.com/KellerJordan/modded-nanogpt>) 社区成员在此分享了许多有趣的改进方法，用于“加速”小规模语言模型预训练。例如，一种常见的改进方法可以追溯到Transformer的原始论文，即把输入和输出嵌入的权重绑定在一起（参见Vaswani等人[2017]（第3.4节）和Chowdhery等人[2022]（第2节））。如果您尝试绑定权重，可能需要降低嵌入/语言模型头初始化的标准差。

在尝试完整的 1.5 小时运行之前，您需要在 OpenWebText 的一小部分或 TinyStories 上测试这些功能。

需要注意的是，您在此排行榜中发现的一些有效修改方法可能并不适用于更大规模的预训练。我们将在课程的“扩展规律”单元中进一步探讨这一问题。

问题（排行榜）：排行榜（6 分）（10 小时 100 小时）

您将按照上述排行榜规则训练一个模型，目标是在 1.5 小时 100 小时内最大限度地减少语言模型的验证损失。

交付成果请提供最终记录的验证损失、清晰显示实际运行时间（x轴）小于1.5小时的相关学习曲线，以及您所做操作的描述。我们希望排行榜提交的作品至少能超越5.0损失的基准值。请在此处提交至排行榜：<https://github.com/stanford-cs336/assignment1-basics-leaderboard>。

参考

Ronen Eldan 和 Yuanzhi Li. TinyStories: 语言模型可以有多小才能仍然表达连贯的内容
英语? , 2023 年. arXiv:2305.07759.

亚伦·戈卡斯兰、万尼亚·科恩、埃莉·帕夫利克和斯蒂芬妮·泰勒克斯。OpenWebText 语料库。http://
Skylion007.github.io/OpenWebTextCorpus2019年。

Rico Sennrich、Barry Haddow 和 Alexandra Birch。“带子词的罕见词的神经机器翻译”
单位。ACL 的进展2016年。

Changhan Wang、Kyunghyun Cho 和 Jiatao Gu,《基于字节级子词的神经机器翻译》,
2019. arXiv:1909.03341。

Philip Gage. 一种新的数据压缩算法。C 用户期刊, 12(2):23–38, 1994年2月。ISSN
0898-9788。

Alec Radford、Jeff Wu、Rewon Child、David Luan、Dario Amodei 和 Ilya Sutskever。语言模型是
无监督多任务学习器, 2019 年。

Alec Radford、Karthik Narasimhan、Tim Salimans 和 Ilya Sutskever。《提高语言理解能力》
通过生成式预训练, 2018 年。

阿什什·瓦斯瓦尼 (Ashish Vaswani)、诺姆·沙泽尔 (Noam Shazeer)、尼基·帕尔玛 (Niki Parmar)、雅各布·乌什科雷特 (Jakob Uszkoreit)、Llion Jones、艾丹·N·戈麦斯 (Aidan N Gomez)、Łukasz Kaiser、
还有伊利亚·波洛苏欣。你只需要集中注意力。神经网络系统会议论文集2017年。

Toan Q. Nguyen 和 Julian Salazar。《无泪变形金刚: 改善自我规范化》
注意。IWSLT 会议论文集2019年。

熊瑞斌、杨运昌、何迪、郑凯、郑树新、陈星、张辉帅、蓝燕燕、
王立伟和刘铁彦。关于Transformer架构中的层归一化。ICML会议论文集2020年。

Jimmy Lei Ba、Jamie Ryan Kiros 和 Geoffrey E. Hinton。层归一化, 2016 年。arXiv:1607.06450。

雨果·图夫龙、蒂博·拉夫里尔、戈蒂埃·伊扎卡尔、泽维尔·马丁内特、玛丽-安妮·拉肖、蒂莫西·拉克鲁瓦、
Baptiste Rozière、Naman Goyal、Eric Hambro、Faisal Azhar、Aurelien Rodriguez、Armand Joulin、
Edouard Grave 和 Guillaume Lample。Llama: 开放且高效的基础语言模型, 2023 年。arXiv:2302.13971。

张彪和Rico Sennrich。均方根层归一化。神经网络系统会议论文集2019年。

Aaron Grattafiori、Abhimanyu Dubey、Abhinav Jauhri、Abhinav Pandey、Abhishek Kadian、Ahmad Al-
Dahle、Aiesha Letman、Akhil Mathur、Alan Schelten、Alex Vaughan、Amy Yang、Angela Fan、Anirudh
Goyal、Anthony Hartshorn、Aobo Yang、Archi Mitra、Archie Sravankumar、Artem Korenev、Arthur
Hinsvark、Arun Rao、Aston Zhang、Aurelien Rodriguez、Austen Gregerson、Ava Spataru、Baptiste罗齐埃、
贝瑟尼·拜伦、平·唐、鲍比·陈、夏洛特·考切特、查亚·纳亚克、克洛伊·毕、克里斯·马拉、克里斯·麦康奈
尔、克里斯蒂安·凯勒、克里斯托弗·图雷、吴春阳、科琳·王、克里斯蒂安·坎顿·费雷尔、赛勒斯·尼古拉迪
斯、达米安·阿隆修斯、丹尼尔·宋、丹妮尔·平茨、丹尼·利夫希茨、丹尼·怀亚特、大卫·埃西奥布、Dhruv乔
杜里、德鲁夫·马哈詹、迭戈加西亚-奥拉诺、迭戈·佩里诺、迪厄克·哈普克斯、叶戈尔·拉科姆金、埃哈布·阿
尔巴达威、埃琳娜·洛巴诺娃、艾米丽·迪南、埃里克·迈克尔·史密斯、菲利普·拉德诺维奇、弗朗西斯科·古兹
曼、弗兰克·张、加布里埃尔·辛奈夫、加布里埃尔·李、乔治亚·刘易斯、安德森·戈文德·塔泰、格雷姆·内
尔、格雷瓜尔·米隆、关庞、吉列姆·库库雷尔、海莉Nguyen、Hannah Korevaar、胡旭、Hugo Touvron、Iliyan
Zarov、Imanol Arrieta Ibarra、Isabel Kloumann、Ishan Misra、Ivan Evtimov、Jack Chang、Jade Copet、
Jaewon Lee、Jan Geffert、Jana Vranes、Jason Park、Jay Mahadeokar、Jeet Shah、Jelmer van der Linde、
Jennifer Billock、Jenny Hong、Jenya Lee、Jeremy Fu、池建峰、建宇

黄、嘉文刘、王杰、余杰曹、乔安娜·比顿、Joe Spisak、Jongsoo Park、Joseph Rocca、Joshua Johnstun、Joshua Saxe、Junteng Jia、Kalyan Vasuden Alwala、Karthik Prasad、Kartikeya Upasani、Kate Plawiak、Ke Li、Kenneth Heafield、Kevin Stone、Khalid El-Arini、Krithika Iyer、Kshitiz Malik、Kuenley Chiu、Kunal Bhalla、Kushal Lakhota、Lauren Rantala-Yearly、Laurens van der Maaten、Lawrence Chen、Liang Tan、Liz Jenkins、Louis Martin、Lovish Madaan、Lubo Malo、Lukas Blecher、Lukas Landzaat、Luke de Oliveira、Madeline Muzzi、Mahesh Pasupuleti、Mannat Singh、Manohar Paluri、马辛·卡达斯、玛丽亚·钦普凯利、马修·奥尔德姆、马蒂厄·丽塔、玛雅·帕夫洛娃、梅兰妮·坎巴杜尔、迈克·刘易斯、斯敏·米特什·库马尔·辛格、莫娜·哈桑、纳曼·戈亚尔、纳尔吉斯·托拉比、尼古拉·巴什利科夫、尼古拉·博戈伊切夫、尼拉德里·查特吉、张宁、奥利维尔·杜兴、奥努尔·切莱比、帕特里克·阿拉西、张鹏川、李鹏伟、Petar Vasic、Peter Weng、Prajwal Bhargava、Pratik Dubal、Praveen Krishnan、Punit Singh Koura、徐朴新、何清、庆晓东、Ragavan Srinivasan、Raj Ganapathy、Ramon Calderer、Ricardo Silveira Cabral、Robert Stojnic、Roberta Raileanu、Rohan Maheswari、Rohit Girdhar、Rohit Patel、Romain索韦斯特、罗尼·波利多罗、罗尚Sumbaly、Ross Taylor、Ruan Silva、侯锐、Rui Wang、Saghar Hosseini、Sahana Chennabasappa、Sanjay Singh、Sean Bell、Seohyun Sonia Kim、Sergey Edunov、聂少良、Sharan Narang、Sharath Raparthy、Sheng Shen、Shengye Wan、Shruti Bhosale、Shun 张、Simon Vandenhende、Soumya Batra、Spencer Whitman、斯坦·苏特拉、史蒂芬·科洛特、苏金·古鲁甘、西德尼·博罗金斯基、塔马尔·赫尔曼、塔拉·福勒、塔里克·谢沙、托马斯·乔治欧、托马斯·夏洛姆、托比亚斯·斯佩克巴赫、托多尔·米哈伊洛夫、童晓、乌吉瓦尔·卡恩、维达努·戈斯瓦米、维博尔·古普塔、维涅什·拉马坦、维克多·科克兹、文森特·贡格特、维吉妮·多、维什·沃盖蒂、Vítor Albiero、Vladan Petrovic、Weiwei Chu、Wenhan Xiong、Wenyin Fu、Whitney Meers、Xavier Martinet、王晓东、王晓芳、谭小青、夏希德、谢新峰、贾旭超、王学伟、Yaelle Goldschlag、Yashesh Gaur、Yasmine Babaei、文怡、宋一文、张雨辰、李悦、毛宇宁、Zacharie Delpierre Coudert、Zheng Yan、Zhengxing Chen、Zoe Papakipos、Aaditya Singh、Aayushi Srivastava、Abha Jain、Adam Kelsey、Adam Shajnfeld、Adithya Gangidi、Adolfo Victoria、Ahuva Goldstand、Ajay Menon、Ajay Sharma、Alex Boesenberg、Alexei Baevski、Allie Feinstein、Amanda Kallet、Amit Sangani、Amos Teo、Anam Yunus、安德烈·卢普、安德烈斯阿尔瓦拉多、安德鲁·卡普尔斯、安德鲁·顾、安德鲁·何、安德鲁·波尔顿、安德鲁·瑞安、安基特·拉姆昌达尼、安妮·东、安妮·弗兰科、阿努吉·戈亚尔、阿帕拉吉塔·萨拉夫、阿尔卡班杜·乔杜里、阿什利·加布里埃尔、阿什温·巴哈姆贝、阿萨夫·艾森曼、阿扎德·亚兹丹、博·詹姆斯、本·毛雷尔、本杰明·莱昂哈迪、伯尼·黄、贝丝·洛伊德、贝托·德保拉、巴尔加维·帕兰贾普、刘冰、吴博、倪博宇、布雷登·汉考克、布拉姆·瓦斯蒂、布兰登·斯宾塞、布拉尼·斯托伊科维奇、布莱恩·加米多、布里特·蒙塔沃、卡尔·帕克、卡莉·伯顿、卡塔利娜·梅希亚、刘策、王昌翰、金昌奎、周超、胡切斯特、朱庆祥、蔡克里斯、克里斯·廷德尔、克里斯托夫·费希滕霍夫、高辛西娅、达蒙·西文、达娜·比蒂、丹尼尔·克雷默、李丹尼尔、大卫·阿德金斯、徐大卫、戴维·特斯图金、德莉娅·戴维、德维·帕里克、戴安娜·利斯科维奇、迪德姆·福斯、王鼎康、乐德、达斯汀·霍兰德、爱德华·道林、艾萨·贾米尔、伊莱恩·蒙哥马利、埃莉奥诺拉·普雷萨尼、艾米丽·哈恩、艾米丽·伍德、埃里克·图安·勒、埃里克·布林克曼、埃斯特班·阿尔考特、埃文·邓巴、埃文·斯莫瑟斯、孙飞、费利克斯·克鲁克、田峰、菲利波斯·科基诺斯、菲拉特·厄兹格内尔、弗朗切斯科·卡乔尼、弗兰克·卡纳耶特、弗兰克·塞德、加布里埃拉·梅迪纳·弗洛雷斯、加布里埃拉·施瓦茨、加达Badeer、Georgia Sweet、Gil Halpern、Grant Herman、Grigory Sizov、Guangyi、Zhang、Guna Lakshminarayanan、Hakan Inan、Hamid Shojanazeri、Han Zou、Hannah Wang、Hanwen Zha、Haroun Habeeb、Harrison Rudolph、Helen Suk、Henry Aspegren、Hunter Goldman、詹宏源、Ibrahim Damlaj、Igor Molybog、Igor图法诺夫、伊利亚斯·莱昂蒂亚迪斯、伊琳娜·埃琳娜·维利切、伊泰·盖特、杰克·韦斯曼、詹姆斯·格博斯基、詹姆斯·科利、珍妮丝·林、贾菲特·阿舍、让-巴蒂斯特·加亚、杰夫·马库斯、杰夫·唐、詹妮弗·陈、珍妮·珍、杰里米·雷森斯坦、杰里米·特博尔、杰西卡·钟、金健、杨静怡、乔·卡明斯、乔恩·卡维尔、乔恩·谢泼德、乔纳森·麦克菲、乔纳森·托雷斯、乔什·金斯伯格、王俊杰、吴凯、Kam Hou U、Karan Saxena、Kartikay Khandelwal、Katayoun Zand、Kathy Matosich、Kaushik Veeraraghavan、Kelly Michelena、李克谦、Kiran Jagadeesh、Kun Huang、Kunal Chawla、Kyle Huang、Lailin Chen、Lakshya Garg、Lavender A、Leandro Silva、Lee Bell、张蕾、郭亮鹏、于立成、利隆·莫什科维奇、卢卡·韦尔斯特、马迪安·卡布萨、马纳夫·阿瓦拉尼、曼尼什·巴特、马丁纳斯·曼库斯、马坦·哈森、马修·伦尼、马蒂亚斯·雷索、马克西姆·格罗舍夫、马克西姆·瑙莫夫、玛雅·拉希、梅根·肯尼利、刘淼、迈克尔·塞尔策、米哈尔·瓦尔科、米歇尔·雷斯特雷波、米希尔·帕特尔、米克·维亚茨科夫、Mikayel Samvelyan、Mike Clark、Mike Macey、Mike Wang、Miquel Jubert Hermoso、Mo Metanat、Moham-杰克·韦斯曼、詹姆斯·格博斯基、詹姆斯·科利、珍妮丝·林、贾菲特·阿舍、让-巴蒂斯特·加亚、杰夫·马库斯、杰夫·唐、詹妮弗·陈、珍妮·珍、杰里米·雷森斯坦、杰里米·特博尔、杰西卡·钟、金健、杨静怡、乔·卡明斯、乔恩·卡维尔、乔恩·谢泼德、乔纳森·麦菲、乔纳森·托雷斯、乔什·金斯伯格、王俊杰、凯Wu、Kam Hou U、Karan Saxena、Kartikay Khandelwal、Katayoun Zand、Kathy Matosich、Kaushik Veeraraghavan、Kelly Michelena、Keqian Li、Kiran Jagadeesh、Kun Huang、Kunal Chawla、Kyle Huang、Lailin Chen、Lakshya Garg、Lavender A、Leandro Silva、Lee Bell、Lei 张、LiangpengGuo、Li Cheng Yu、Liron Moshkovich、卢卡·韦尔施泰特、马迪安卡布萨、马纳夫·阿瓦拉尼、曼尼什·巴特、马蒂纳斯·曼库斯、马坦·哈森、马修·伦尼、马蒂亚斯·雷索、马克西姆·格罗舍夫、马克西姆·瑙莫夫、玛雅·拉希、梅根·肯尼利、刘淼、迈克尔·L·塞尔策、米哈尔·瓦尔科、米歇尔·雷斯特雷波、米希尔·帕特尔、米克·维亚茨科夫、米凯尔·萨姆韦利安、迈克·克拉克、迈克·梅西、迈克·王、米克尔·尤伯特·埃尔莫索、莫·梅塔纳特、莫罕-杰克·韦斯曼、詹姆斯·格博斯基、詹姆斯·科利、珍妮丝·林、贾菲特·阿舍、让-巴蒂斯特·加亚、杰夫·马库斯、杰夫·唐、詹妮弗·陈、珍妮·珍、杰里米·雷森斯坦、杰里米·特博尔、杰西卡·钟、金健、杨静怡、乔·卡明斯、乔恩·卡维尔、乔恩·谢泼德、乔纳森·麦菲、乔纳森·托雷斯、乔什·金斯伯格、王俊杰、凯Wu、Kam Hou U、Karan Saxena、Kartikay Khandelwal、Katayoun Zand、Kathy Matosich、Kaushik Veeraraghavan、Kelly Michelena、Keqian Li、Kiran Jagadeesh、Kun Huang、Kunal Chawla、Kyle Huang、Lailin Chen、Lakshya Garg、Lavender A、Leandro Silva、Lee Bell、Lei 张、LiangpengGuo、Li Cheng Yu、Liron Moshkovich、卢卡·韦尔施泰特、马迪安卡布萨、马纳夫·阿瓦拉尼、曼尼什·巴特、马蒂纳斯·曼库斯、马坦·哈森、马修·伦尼、马蒂亚斯·雷索、马克西姆·格罗舍夫、马克西姆·瑙莫夫、玛雅·拉希、梅根·肯尼利、刘淼、迈克尔·L·塞尔策、米哈尔·瓦尔科、米歇尔·雷斯特雷波、米希尔·帕特尔、米克·维亚茨科夫、米凯尔·萨姆韦利安、迈克·克拉克、迈克·梅西、迈克·王、米克尔·尤伯特·埃尔莫索、莫·梅塔纳特、莫罕-

mad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, 宁东, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, 钱亮, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, 丽贝卡·霍根, 罗宾·巴蒂, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy 查, Shishir Patil, Shiva Shankar, 张书强, 张书强, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal 塔玛拉·雷梅兹, 塔玛·格拉泽 Best, Thilo Koehler, Thomas Robinson, 李天河, 张天军, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, 唐晓成, 吴晓建, 王晓兰, 吴希伦, 高新波, Yaniv Kleinman, 陈彦君, 胡野, 叶嘉, 叶琪, Yenda Li, 张一琳, 张英, Yossi Adi, Youngjin Nam, 于, 王, 赵宇, 郝雨辰, 钱云迪, 李云路, 何宇子, Zach Rait, Zachary DeVito, Zef Rosnbrick, 温兆铎, 杨振宇, 赵志伟, 马志宇. 美洲驼 3 群模特, 2024 年. URL <https://arxiv.org/abs/2407.21783>.

安阳, 杨宝松, 张北辰, 惠斌源, 郑波, 余博文, 李成远, 刘大一恒, 黄飞, 魏浩然, 林焕, 杨健, 涂建宏, 张建伟, 杨建新, 杨家喜, 周静仁, 林俊阳, 党凯, 卢克明, 包克勤, 杨克新, 于乐, 李梅, 薛明峰, 张培, 朱勤, 门瑞, 林润吉, 李天浩, 夏廷宇, 任兴章, 任宣成, 杨帆, 苏扬, 张一昌, 万宇, 玉琼刘, 崔泽宇, 张真如, 邱子涵. Qwen2.5 技术报告. *arXiv 预印本 arXiv:2412.15115* 2024 年.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, 保罗·巴勒姆, 郑亨元, 查尔斯·萨顿, 塞巴斯蒂安·格尔曼, 帕克·舒, 施肯森, 萨沙·茨维亚先科, 约书亚·梅内斯, 阿布舍克·拉奥, 帕克·巴恩斯, Yi Tay, 诺姆·沙泽尔, 维诺库马尔·普拉巴卡兰, 艾米丽·莱夫, Nan Du, 本·哈钦森, Reiner Pope, 詹姆斯·布拉伯里, 雅各布·奥斯汀, 迈克尔·伊萨德, Guy Gur-Ari, Peng Cheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark 奥默尼克, 安德鲁·M·戴, 塔努马拉扬 Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, 周宗伟, 王学智, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov 和 Noah Fiedel. PaLM: 通过路径扩展语言建模, 2022. *arXiv: 2204.02311*.

Dan Hendrycks 和 Kevin Gimpel. 利用高斯误差连接非线性和随机正则化项线性单位, 2016 年. *arXiv:1606.08415*.

Stefan Elfwing, Eiji Uchibe 和 Kenji Doya. 用于神经网络函数的 Sigmoid 加权线性单元强化学习中的近似, 2017. 网址 <https://arxiv.org/abs/1702.03118>.

Yann N. Dauphin, Angela Fan, Michael Auli 和 David Grangier. 使用门控卷积的语言建模国家网络, 2017 年. 网址 <https://arxiv.org/abs/1612.08083>.

Noam Shazeer. GLU 变体改进 transformer, 2020. *arXiv:2002.05202*.

苏建林, 卢宇, 潘胜峰, 文博, 刘云峰. Roformer: 带旋转的增强型变压器位置嵌入, 2021.

Diederik P. Kingma 和 Jimmy Ba. Adam：一种随机优化方法。 *ICLR会议论文集*2015年。

Ilya Loshchilov 和 Frank Hutter. 解耦权重衰减正则化。 *ICLR会议论文集*2019年。

汤姆·B·布朗、本杰明·曼、尼克·莱德、梅兰妮·苏比亚、贾里德·卡普兰、普拉芙拉·达里瓦尔、阿尔温德·尼-
Lakantan、Pranav Shyam、Girish Sastry、Amanda Aspell、Sandhini Agarwal、Ariel Herbert-Voss、Gretchen
Krueger、Tom Henighan、Rewon Child、Aditya Ramesh、Daniel M. Ziegler、Jeffrey Wu、Clemens Winter、
Christopher Hesse、Mark Chen、Eric Sigler、Mateusz Litwin、Scott Gray、Benjamin Chess、Jack Clark、
Christopher Berner、Sam McCandlish、Alec Radford、Ilya Sutskever 和 Dario Amodei。“语言模型是少样本学
习器。” *神经脉冲系统会议论文集*2020年。

贾里德·卡普兰、萨姆·麦坎德利什、汤姆·亨尼根、汤姆·B·布朗、本杰明·切斯、雷翁·查尔德、斯科特
Gray、Alec Radford、Jeffrey Wu 和 Dario Amodei。神经语言模型的缩放定律，2020 年。
arXiv:2001.08361。

Jordan Hoffmann、Sebastian Borgeaud、Arthur Mensch、Elena Buchatskaya、Trevor Cai、Eliza Rutherford
Diego de Las Casas、Lisa Anne Hendricks、Johannes Welbl、Aidan Clark、Tom Hennigan、Eric
Noland、Katie Millican、George van den Driessche、Bogdan Damoc、Aurelia Guy、Simon Osindero、
Karen Simonyan、Erich Elsen、Jack W. Rae、Oriol Vinyals 和 Laurent Sifre。训练计算最优的大型语言模
型，2022 年。arXiv:2203.15556。

Ari Holtzman、Jan Buys、Li Du、Maxwell Forbes 和 Yejin Choi。神经文本退化的奇特案例——
tion。在 *ICLR会议论文集*2020年。

蔡耀宏、白少杰、山田诚、路易斯-菲利普·莫伦西和鲁斯兰·萨拉胡迪诺夫。

Transformer 解构：通过内核视角对 Transformer 的注意力进行统一理解。见 Kentaro Inui、Jing Jiang、
Vincent Ng 和 Xiaojun Wan 编辑的，2019 年自然语言处理实证方法会议暨第九届自然语言处理国际联合会会议
论文集 (*EMNLP-IJCNLP*)，第 4344–4353 页，中国香港，2019 年 11 月。计算语言学协会。doi: 10.18653/v1/
D19-1443。网址<https://aclanthology.org/D19-1443/>。

Amirhossein Kazemnejad、Inkit Padhi、Karthikeyan Natesan、Payel Das 和 Siva Reddy。的影响
Transformer 中长度泛化的位置编码。第三十七届神经信息处理系统会议2023年。网址<https://openreview.net/forum?id=Drrl2gcjzl>。